

In [3]: *# Q1.1 Understanding the Data and Simple Curve Fitting*

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split

# Load dataset
data = pd.read_csv("Advertising-Dataset.csv")

# Features and target
X = data.drop(columns=["Sales"])
y = data["Sales"]

# Train-test split (80-20)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=21)

# Scatter plots of features vs Sales
for col in X_train.columns:
    plt.figure()
    plt.scatter(X_test[col], y_test, marker="^", c="darkorange", alpha=0.7, s=55, label="Test")
    plt.scatter(X_train[col], y_train, marker="o", c="steelblue", alpha=0.7, s=45, label="Train")
    plt.xlabel(col)
    plt.ylabel("Sales")
    plt.title(f"Sales vs {col} (Train & Test)")
    plt.legend(loc="upper left", bbox_to_anchor=(1,1))
    plt.grid(alpha=0.3)
    plt.show()

# Q1.2 Polynomial Curve Fitting with Gradient Descent

import numpy as np

# Build polynomial feature matrix
def poly_features(x, degree):
    x = x.reshape(-1, 1)
    return np.concatenate([x**p for p in range(degree+1)], axis=1)

# Train using gradient descent
def fit_poly_gd(x, y, degree, lr=0.05, epochs=4000):
    xmin, xmax = x.min(), x.max()
    x_norm = (x - xmin) / (xmax - xmin + 1e-12)
    Phi = poly_features(x_norm, degree)
    w = np.zeros(Phi.shape[1])
    m = len(y)
```

```

for _ in range(epochs):
    y_pred = Phi @ w
    grad = (Phi.T @ (y_pred - y)) / m
    w -= lr * grad
return w, (xmin, xmax)

# Predict function
def predict_poly(x, w, bounds, degree):
    xmin, xmax = bounds
    x_norm = (x - xmin) / (xmax - xmin + 1e-12)
    Phi = poly_features(x_norm, degree)
    return Phi @ w

# Error function
def mse(y, yhat):
    return ((y - yhat)**2).mean() / 2

for col in X_train.columns:
    xtr, xte = X_train[col].to_numpy(), X_test[col].to_numpy()
    ytr, yte = y_train.to_numpy(), y_test.to_numpy()

    tr_errs, te_errs = [], []

    for deg in range(1, 10):
        w, norm = fit_poly_gd(xtr, ytr, deg)
        ytr_pred = predict_poly(xtr, w, norm, deg)
        yte_pred = predict_poly(xte, w, norm, deg)

        tr_errs.append(mse(ytr, ytr_pred))
        te_errs.append(mse(yte, yte_pred))

    # Curve plot
    xs = np.linspace(xtr.min(), xtr.max(), 300)
    ys = predict_poly(xs, w, norm, deg)

    plt.figure()
    plt.scatter(xtr, ytr, c="navy", s=35, label="Train", alpha=0.65)
    plt.scatter(xte, yte, c="crimson", s=45, marker="x", label="Test", alpha=0.75)
    plt.plot(xs, ys, linestyle="dashdot", color="darkgreen", label=f"deg {deg}")
    plt.xlabel(col)
    plt.ylabel("Sales")
    plt.title(f"Polynomial Fit of degree {deg} for {col}")
    plt.legend()
    plt.show()

```

```

# Error vs degree plot
degrees = np.arange(1, 10)
best_deg = degrees[np.argmin(te_errs)]

plt.figure(figsize=(6, 4))
plt.plot(degrees, tr_errs, "-o", label="Train MSE")
plt.plot(degrees, te_errs, "--s", label="Test MSE")
plt.axvline(best_deg, color="gray", linestyle=":", label=f"Best = {best_deg}")
plt.xlabel("Polynomial Degree")
plt.ylabel("MSE")
plt.title(f"Training vs Testing Error for {col}")
plt.legend()
plt.grid(alpha=0.4)
plt.show()

print(f"Results for {col}:")
for d, (tr, te) in enumerate(zip(tr_errs, te_errs), start=1):
    print(f"Degree {d}: Train={tr:.4f}, Test={te:.4f}")

```

Q1.3 Regularization (Lasso & Ridge)

Lasso

```

def fit_lasso(x, y, degree, lam=0.0, lr=0.01, epochs=3000):
    xmin, xmax = x.min(), x.max()
    x_norm = (x - xmin) / (xmax - xmin + 1e-8)
    Phi = poly_features(x_norm, degree)
    w = np.zeros(Phi.shape[1])
    m = len(y)

    for _ in range(epochs):
        pred = Phi @ w
        grad = (Phi.T @ (pred - y)) / m
        grad[1:] += lam * np.sign(w[1:])
        w -= lr * grad
    return w, (xmin, xmax)

```

Ridge

```

def fit_ridge(x, y, degree, lam=0.0, lr=0.01, epochs=3000):
    xmin, xmax = x.min(), x.max()
    x_norm = (x - xmin) / (xmax - xmin + 1e-8)
    Phi = poly_features(x_norm, degree)
    w = np.zeros(Phi.shape[1])
    m = len(y)

    for _ in range(epochs):

```

```

    pred = Phi @ w
    grad = (Phi.T @ (pred - y)) / m
    grad[1:] += lam * w[1:]
    w -= lr * grad
    return w, (xmin, xmax)

```

Apply on TV feature

```

tv_train, tv_test = X_train["TV"].to_numpy(), X_test["TV"].to_numpy()
sales_train, sales_test = y_train.to_numpy(), y_test.to_numpy()

```

```

for deg in [2, 3]:
    lambdas = [0, 0.25, 0.5, 0.75, 1.0]

```

Lasso

```

lasso_tr, lasso_te = [], []
for lam in lambdas:
    w, norm = fit_lasso(tv_train, sales_train, deg, lam=lam)
    lasso_tr.append(mse(sales_train, predict_poly(tv_train, w, norm, deg)))
    lasso_te.append(mse(sales_test, predict_poly(tv_test, w, norm, deg)))

```

```

plt.figure(figsize=(6,4))
plt.plot(lambdas, lasso_tr, "-o", label="Train MSE")
plt.plot(lambdas, lasso_te, "--s", label="Test MSE")
plt.title(f"Lasso Regularization (degree={deg})")
plt.xlabel("λ")
plt.ylabel("MSE")
plt.legend()
plt.grid(alpha=0.4)
plt.show()

```

Ridge

```

ridge_tr, ridge_te = [], []
for lam in lambdas:
    w, norm = fit_ridge(tv_train, sales_train, deg, lam=lam)
    ridge_tr.append(mse(sales_train, predict_poly(tv_train, w, norm, deg)))
    ridge_te.append(mse(sales_test, predict_poly(tv_test, w, norm, deg)))

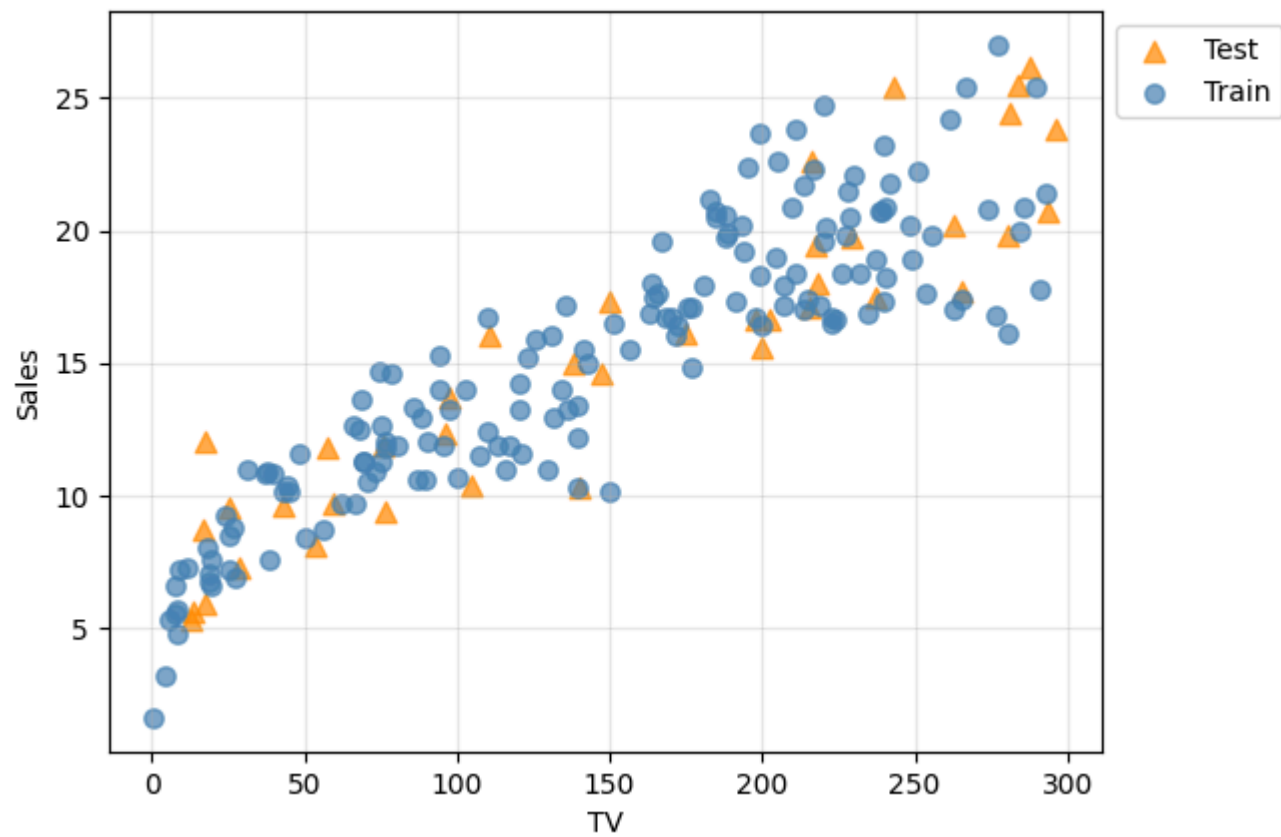
```

```

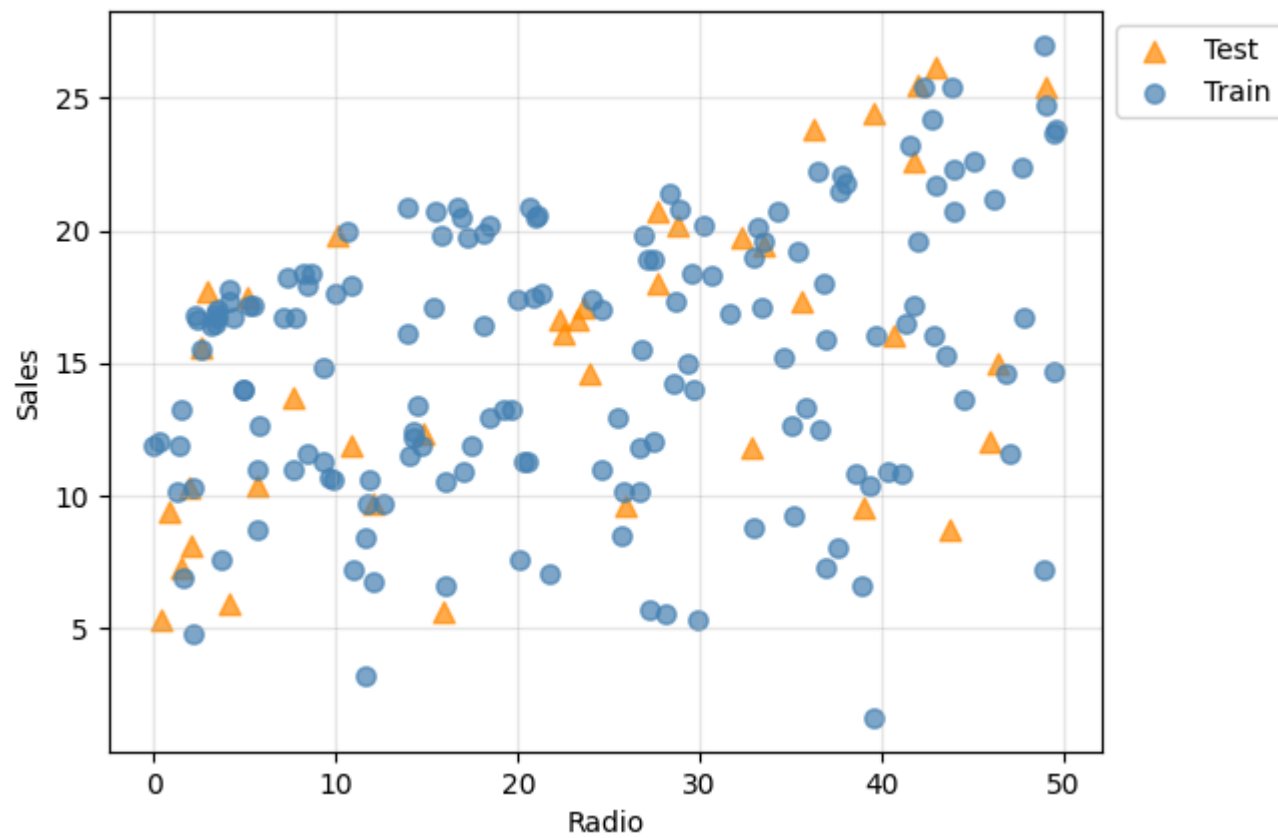
plt.figure(figsize=(6,4))
plt.plot(lambdas, ridge_tr, "-o", label="Train MSE")
plt.plot(lambdas, ridge_te, "--s", label="Test MSE")
plt.title(f"Ridge Regularization (degree={deg})")
plt.xlabel("λ")
plt.ylabel("MSE")
plt.legend()
plt.grid(alpha=0.4)
plt.show()

```

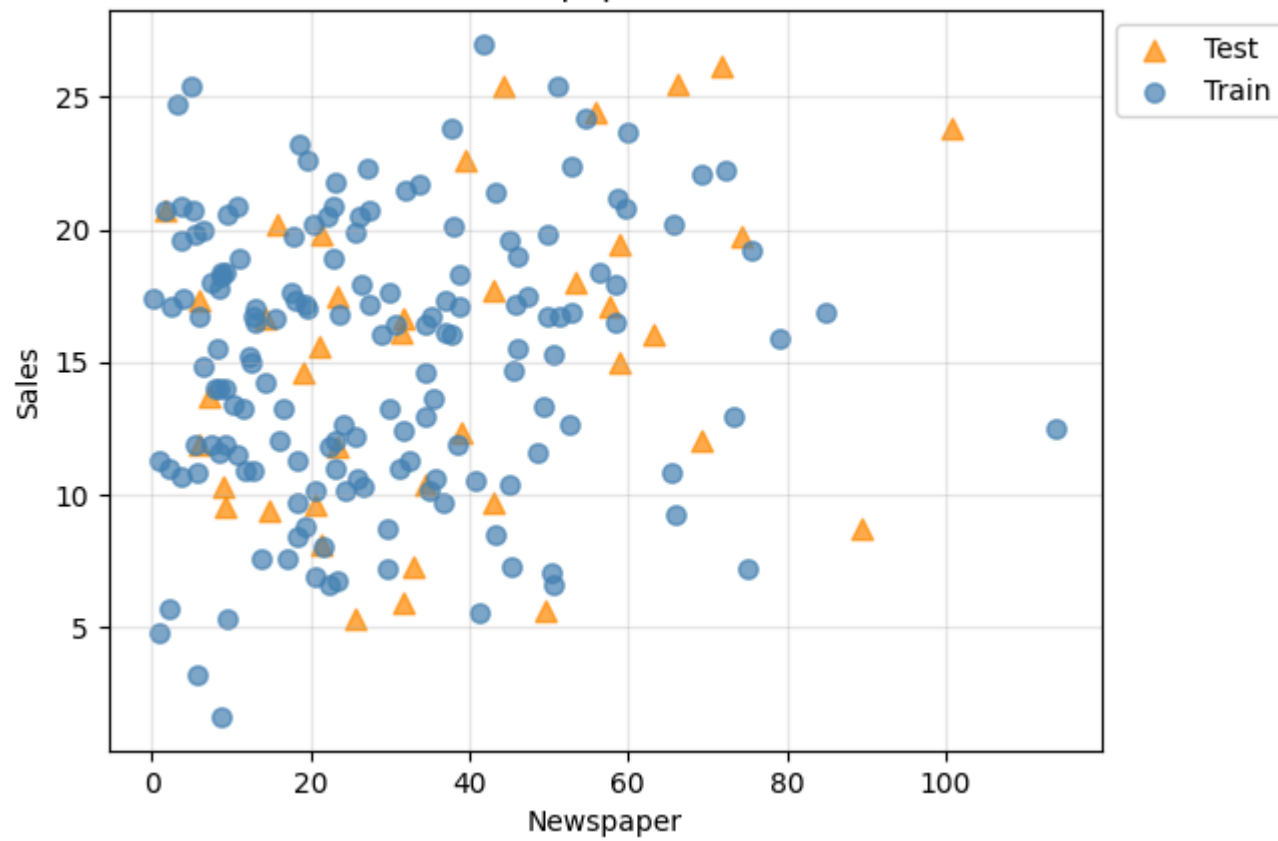
Sales vs TV (Train & Test)



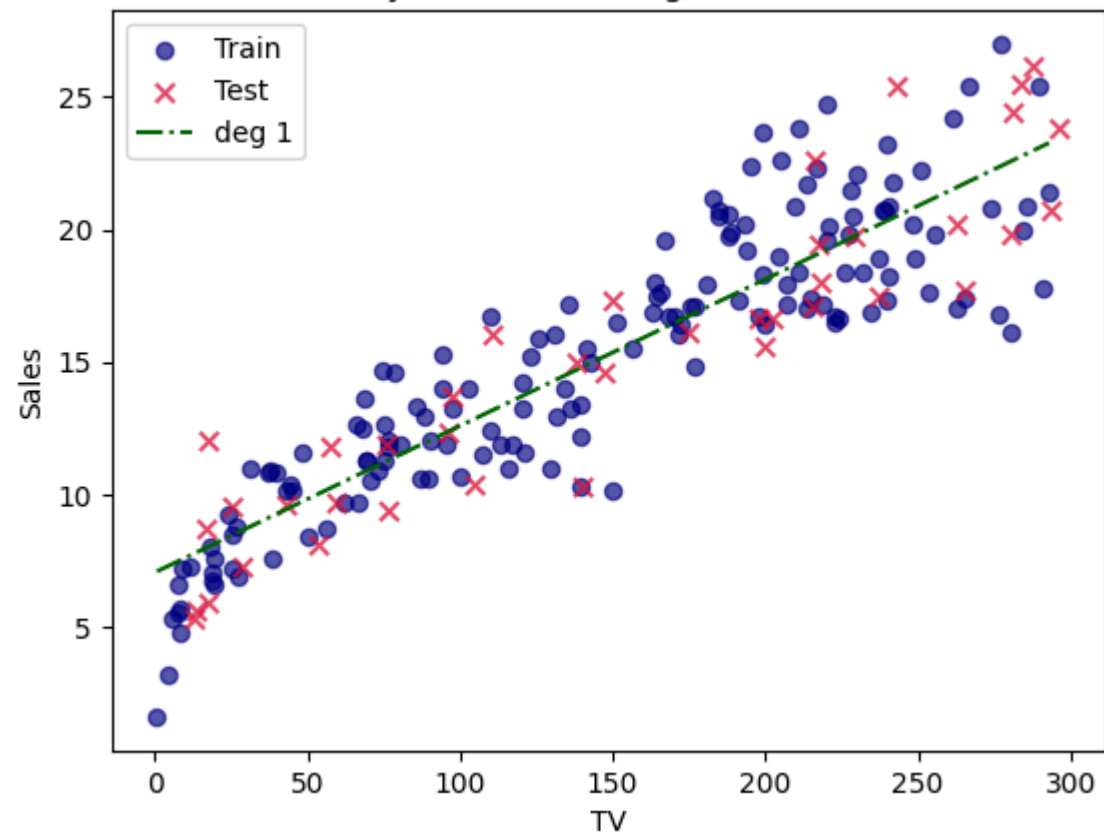
Sales vs Radio (Train & Test)



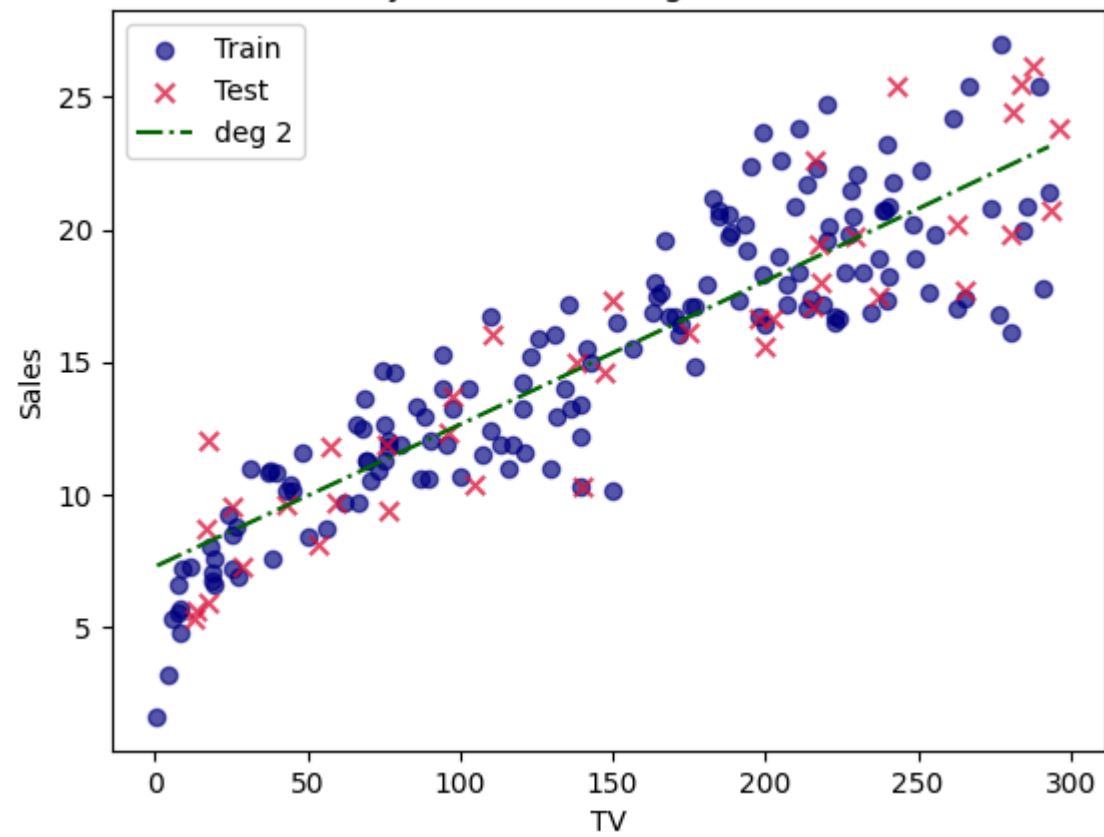
Sales vs Newspaper (Train & Test)



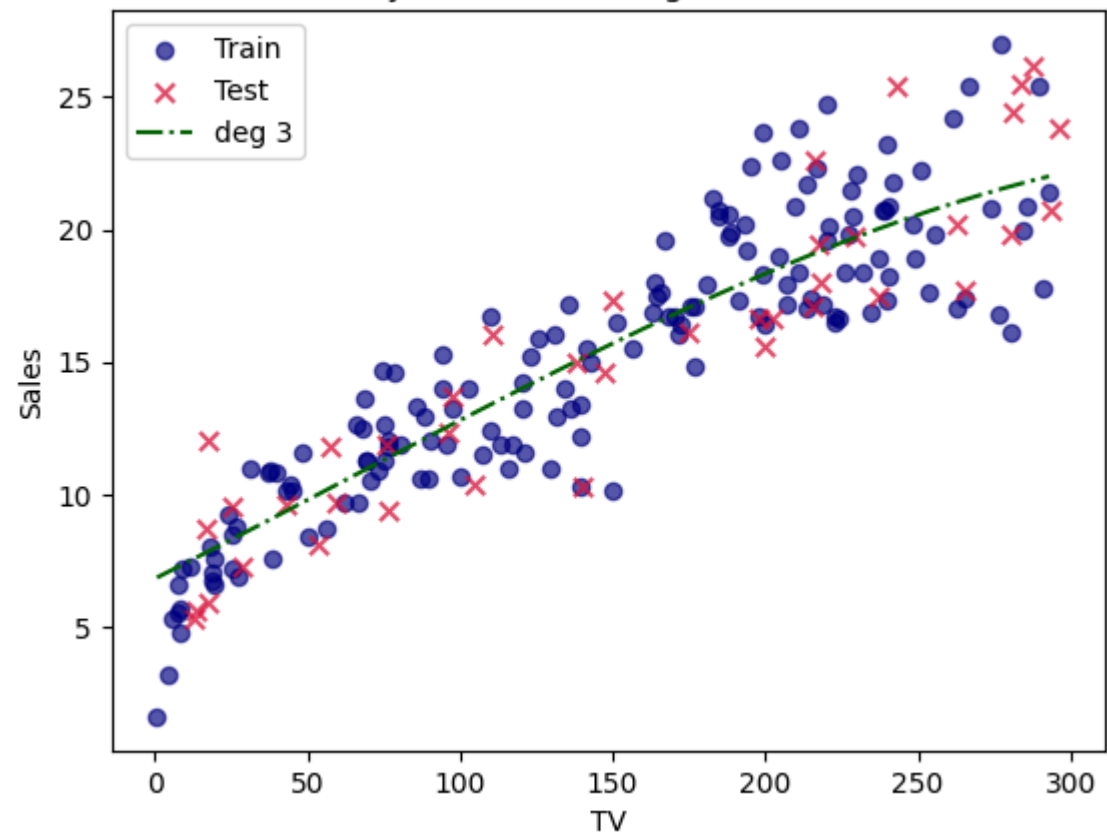
Polynomial Fit of degree 1 for TV



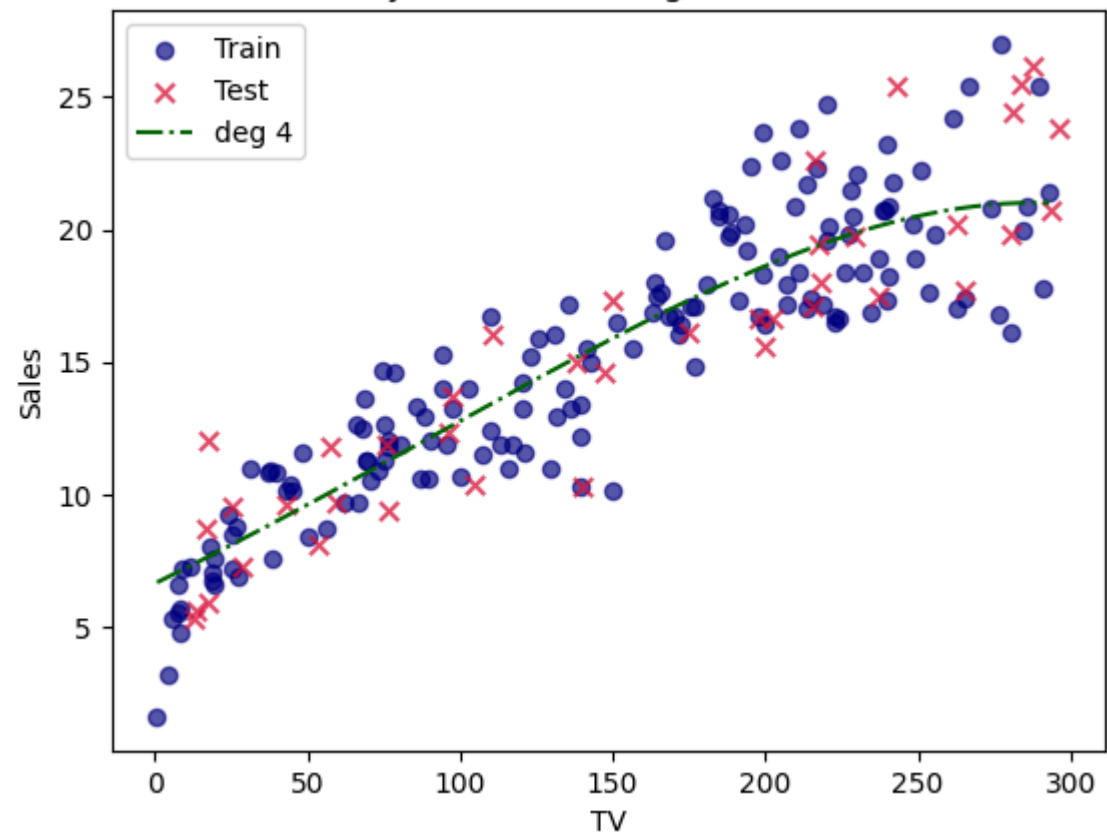
Polynomial Fit of degree 2 for TV



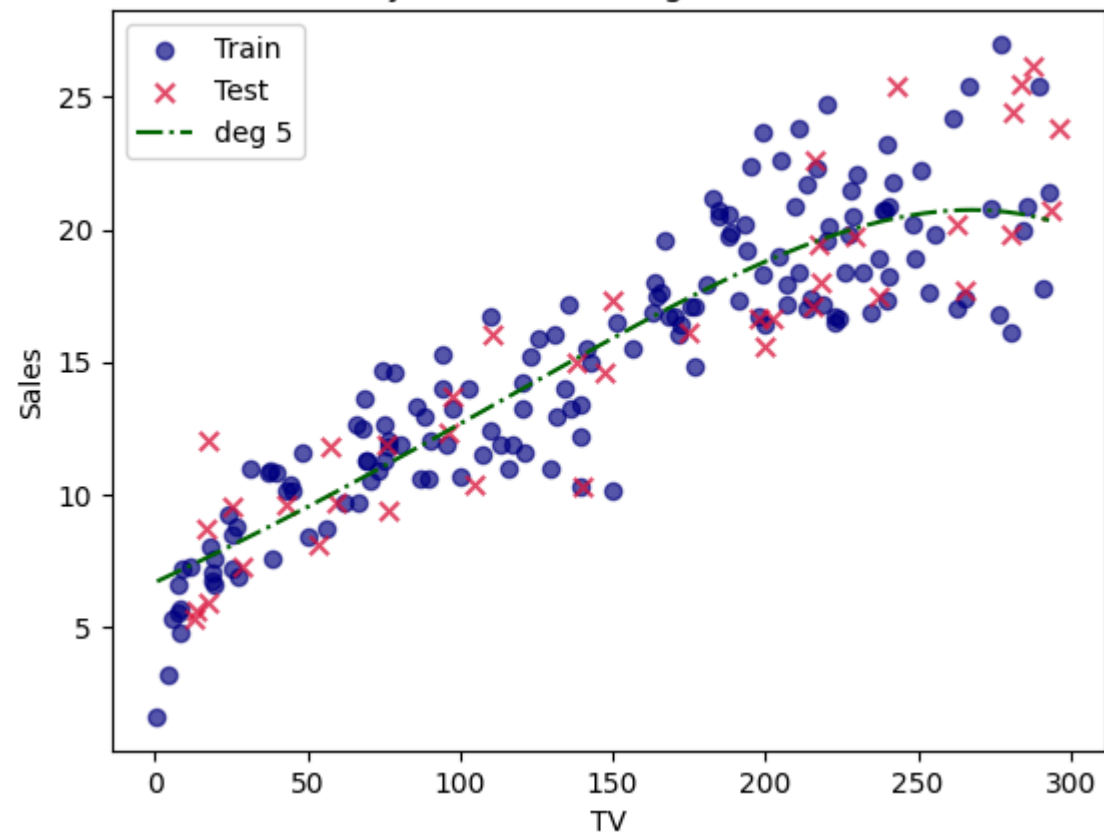
Polynomial Fit of degree 3 for TV



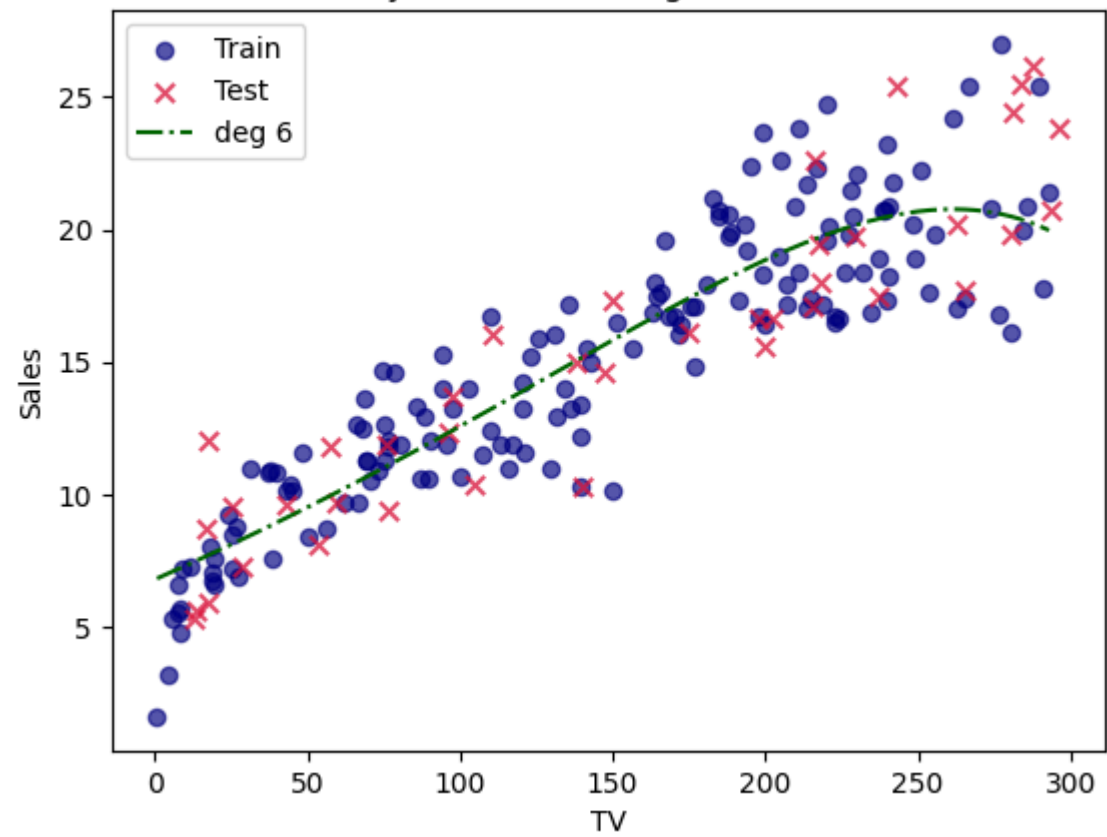
Polynomial Fit of degree 4 for TV



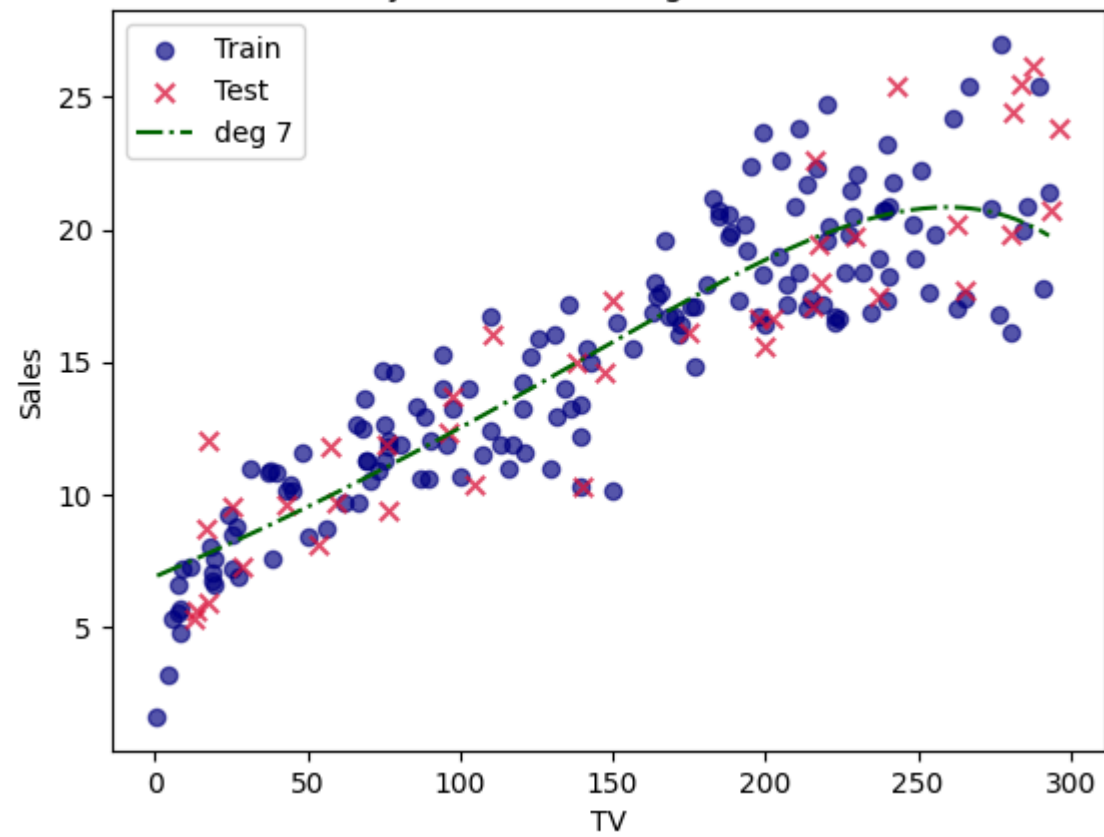
Polynomial Fit of degree 5 for TV



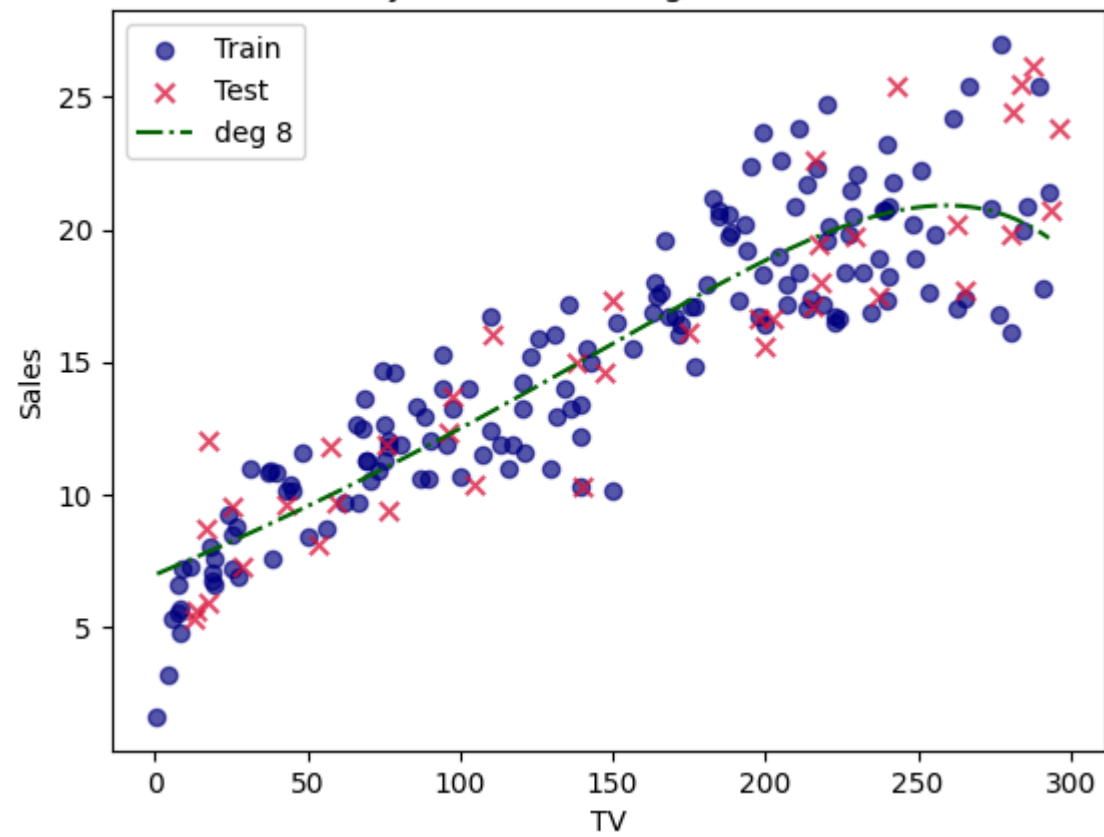
Polynomial Fit of degree 6 for TV



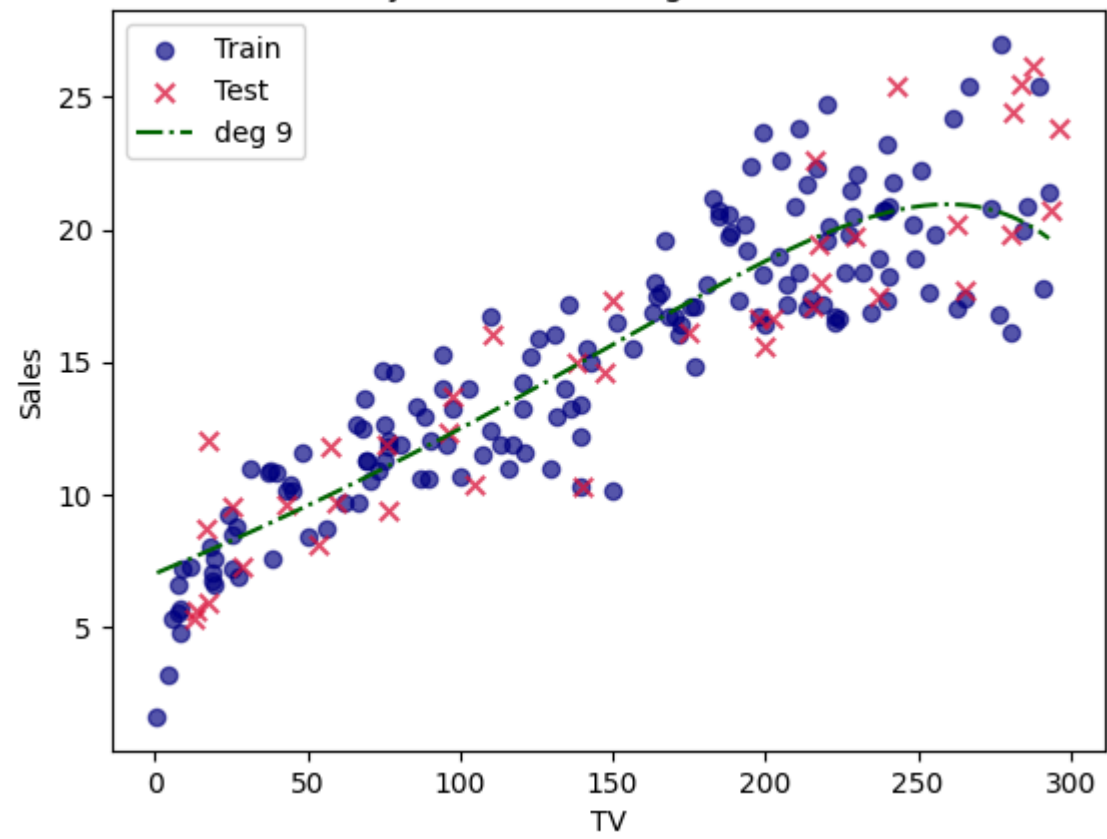
Polynomial Fit of degree 7 for TV



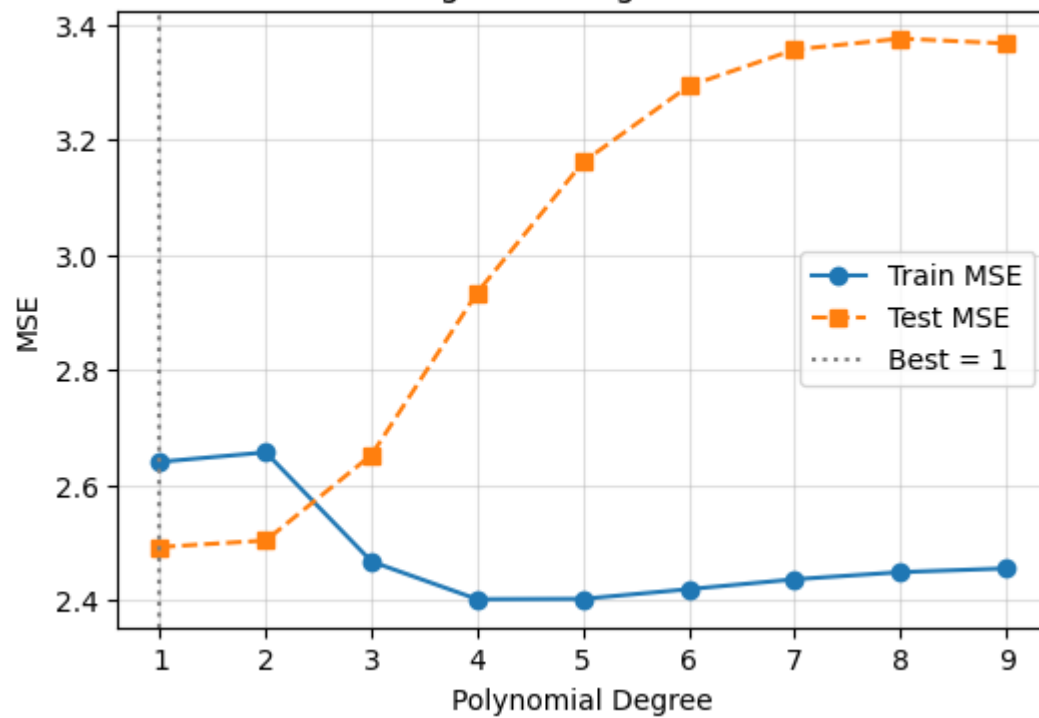
Polynomial Fit of degree 8 for TV



Polynomial Fit of degree 9 for TV



Training vs Testing Error for TV



Results for TV:

Degree 1: Train=2.6402, Test=2.4921

Degree 2: Train=2.6567, Test=2.5034

Degree 3: Train=2.4668, Test=2.6523

Degree 4: Train=2.4012, Test=2.9348

Degree 5: Train=2.4018, Test=3.1616

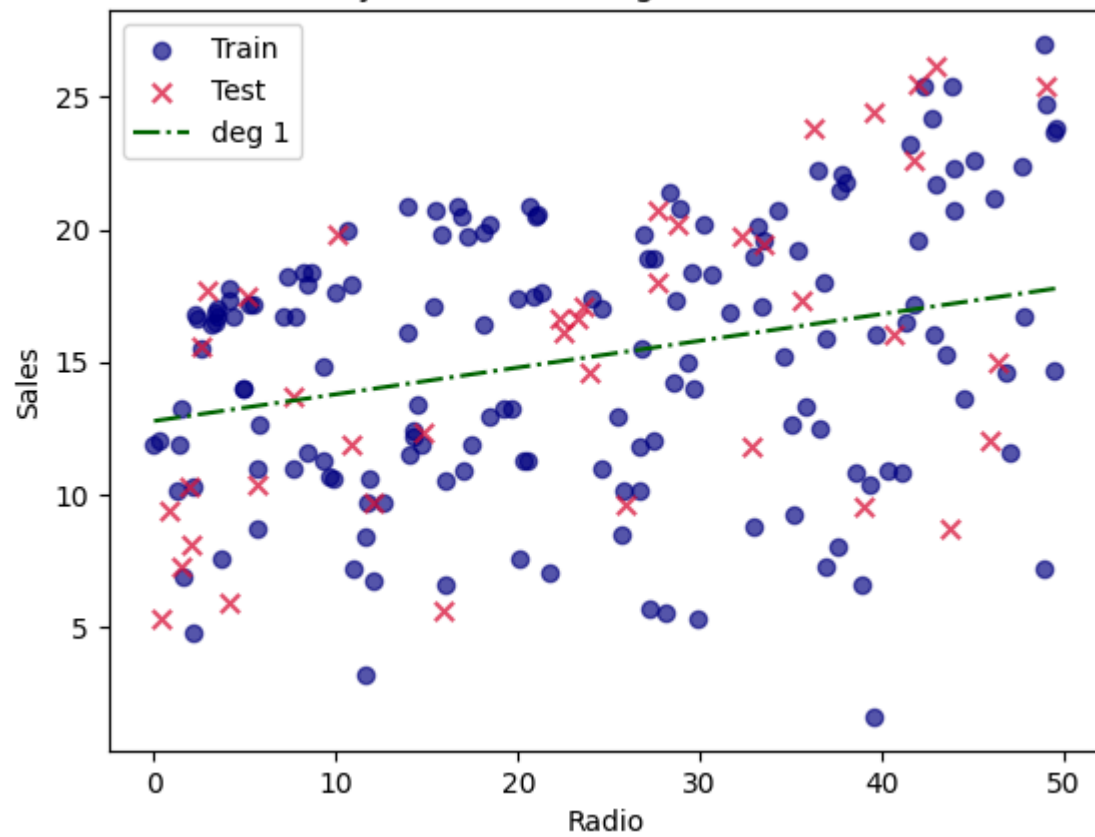
Degree 6: Train=2.4188, Test=3.2939

Degree 7: Train=2.4359, Test=3.3572

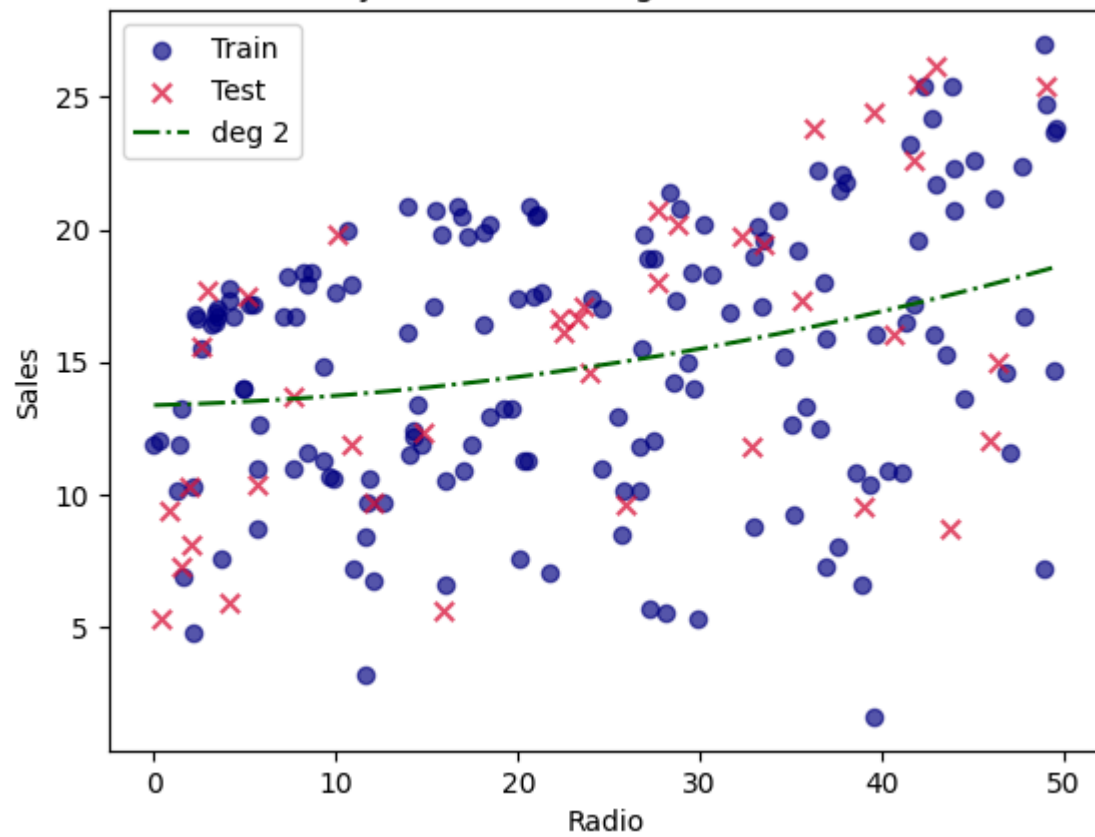
Degree 8: Train=2.4483, Test=3.3764

Degree 9: Train=2.4548, Test=3.3678

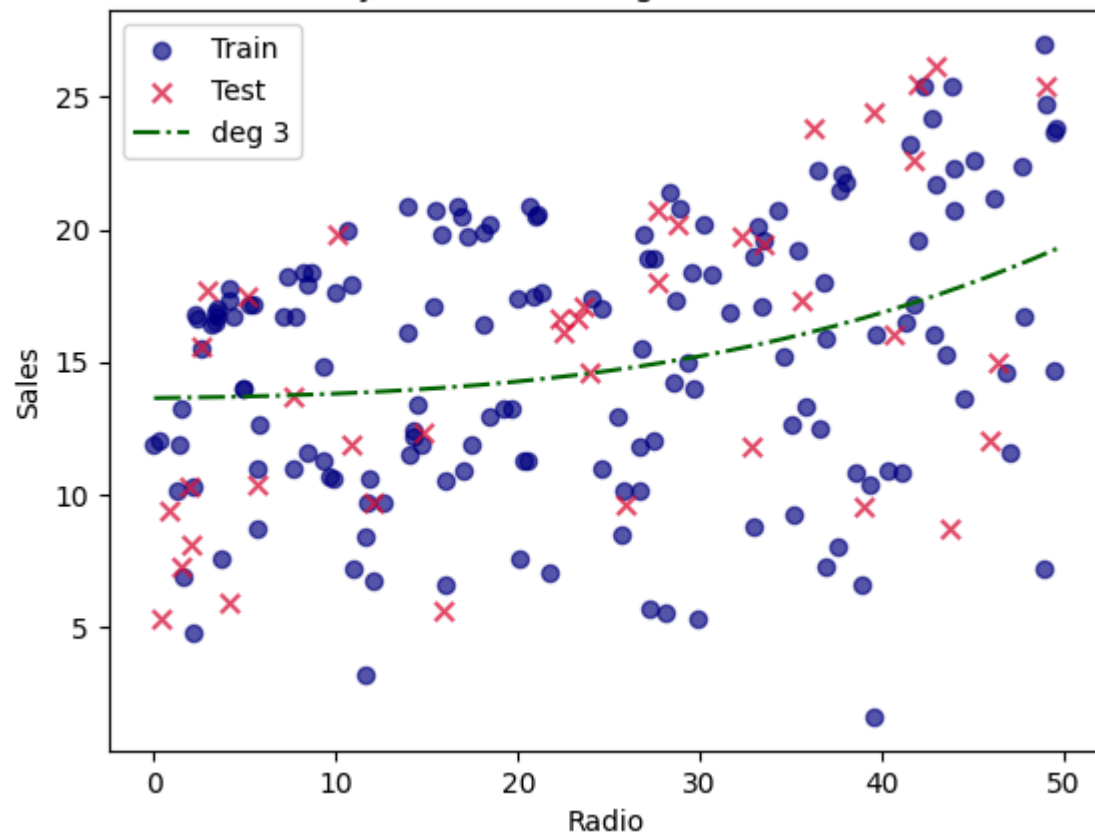
Polynomial Fit of degree 1 for Radio



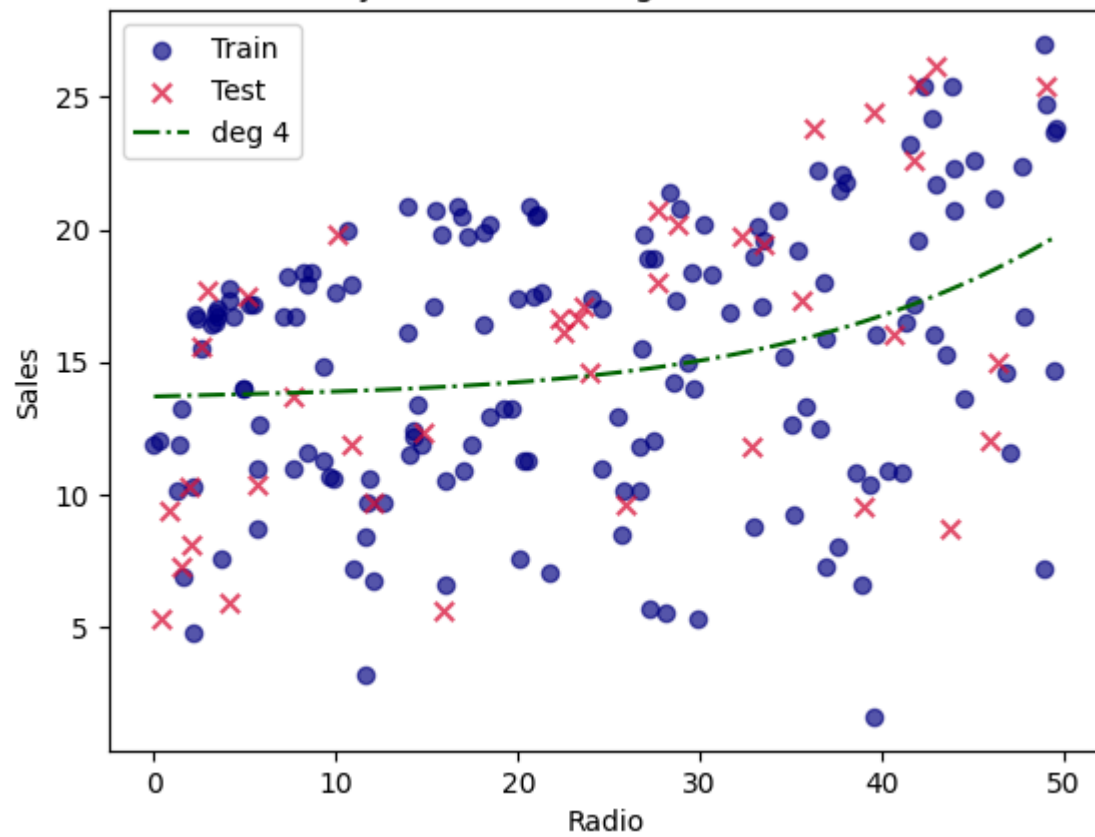
Polynomial Fit of degree 2 for Radio



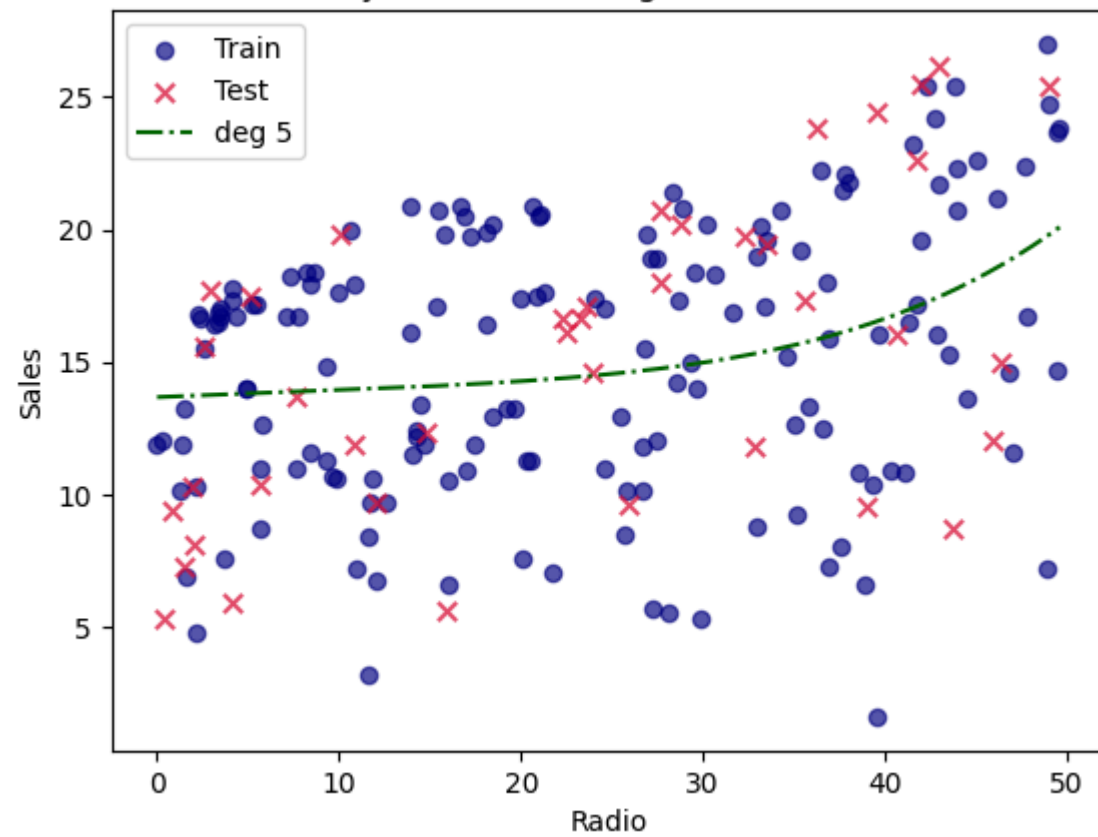
Polynomial Fit of degree 3 for Radio



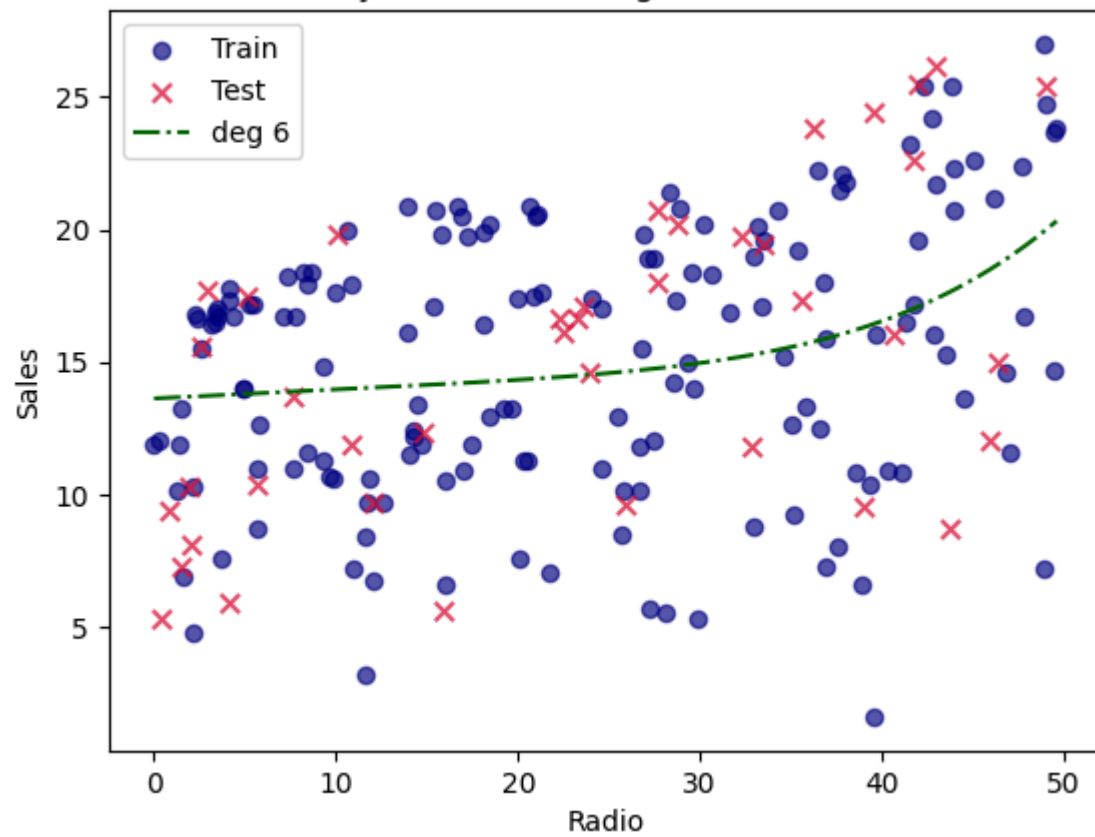
Polynomial Fit of degree 4 for Radio



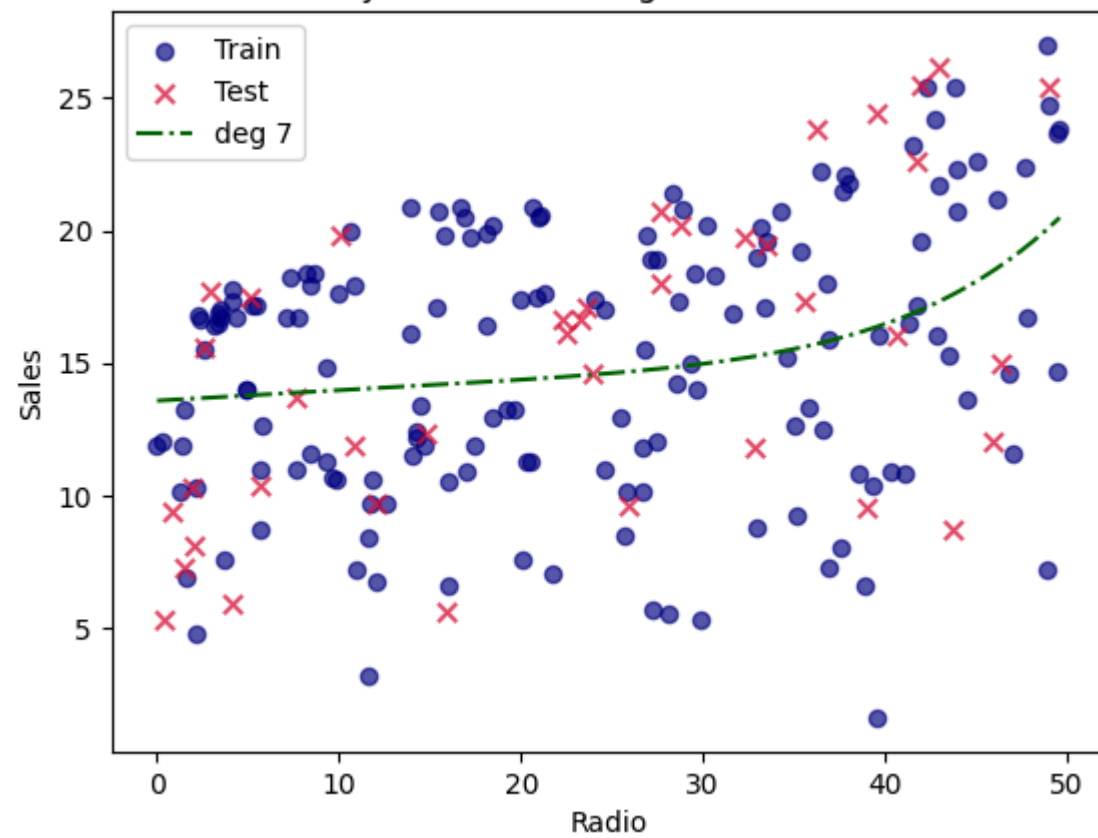
Polynomial Fit of degree 5 for Radio



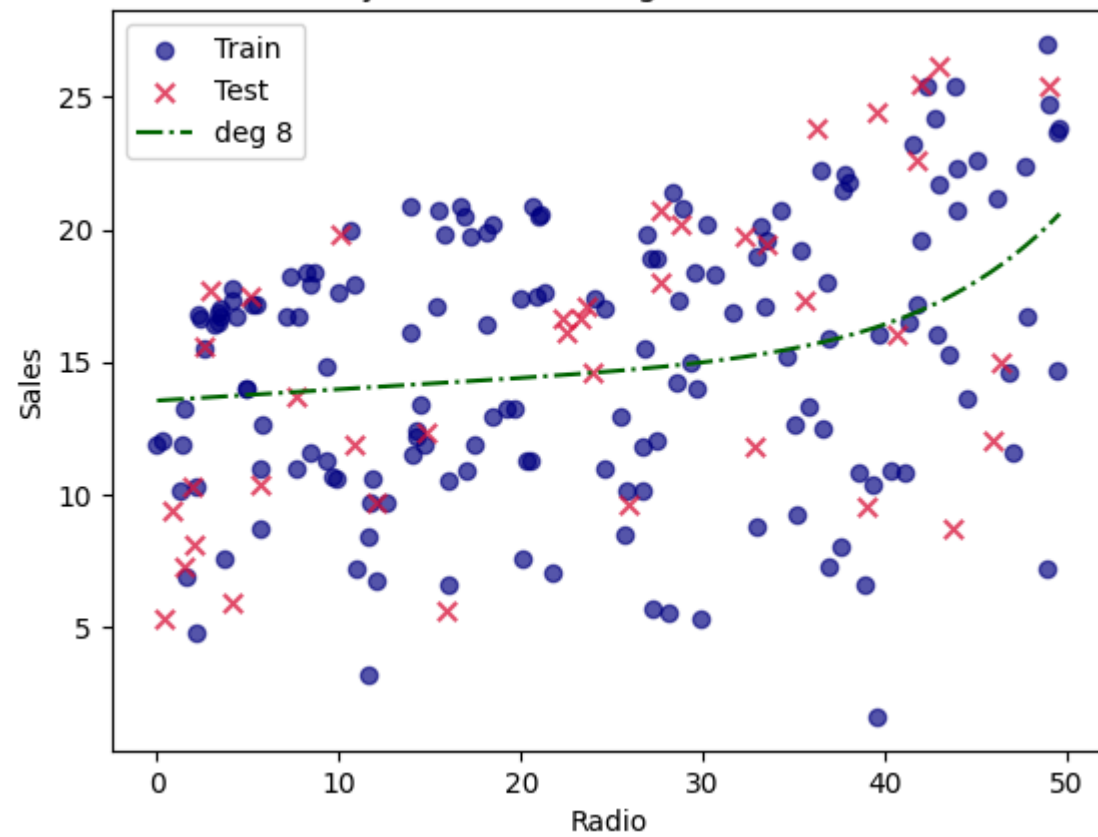
Polynomial Fit of degree 6 for Radio



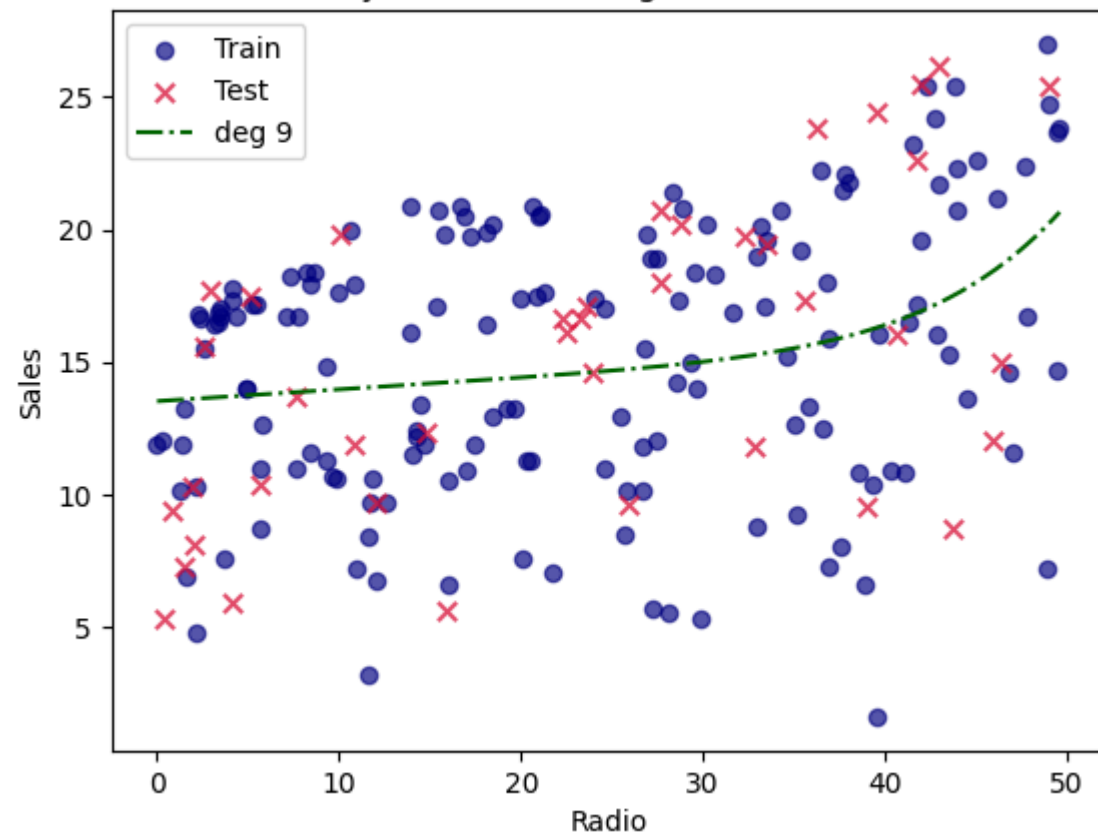
Polynomial Fit of degree 7 for Radio



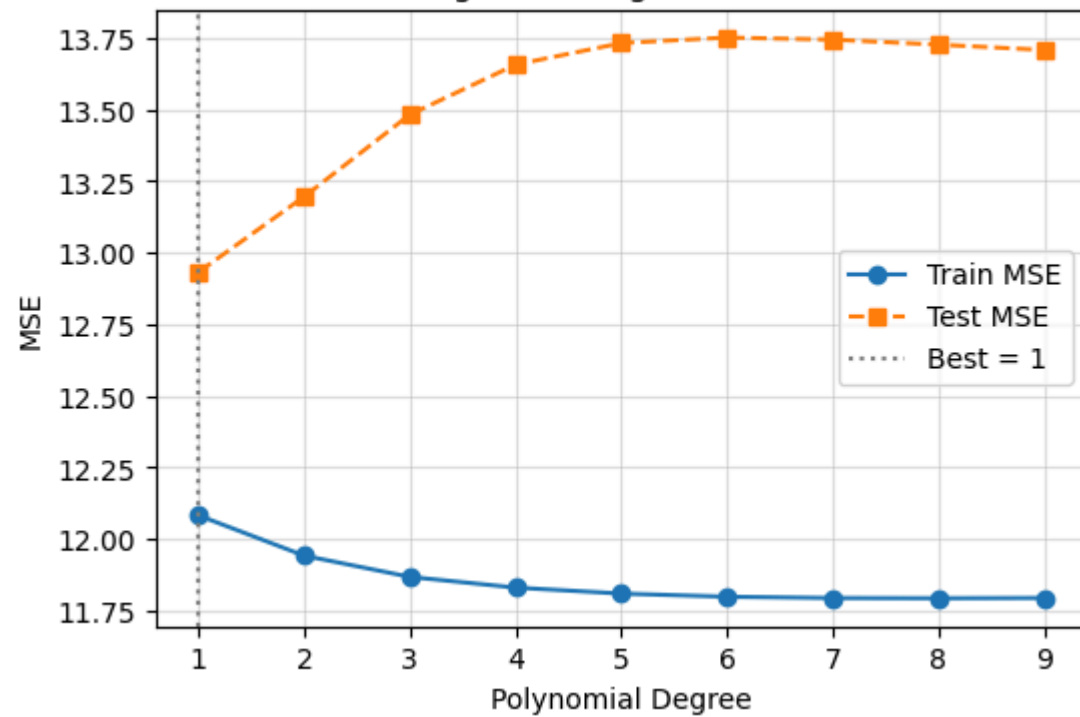
Polynomial Fit of degree 8 for Radio



Polynomial Fit of degree 9 for Radio



Training vs Testing Error for Radio



Results for Radio:

Degree 1: Train=12.0828, Test=12.9317

Degree 2: Train=11.9420, Test=13.1956

Degree 3: Train=11.8678, Test=13.4823

Degree 4: Train=11.8303, Test=13.6569

Degree 5: Train=11.8095, Test=13.7324

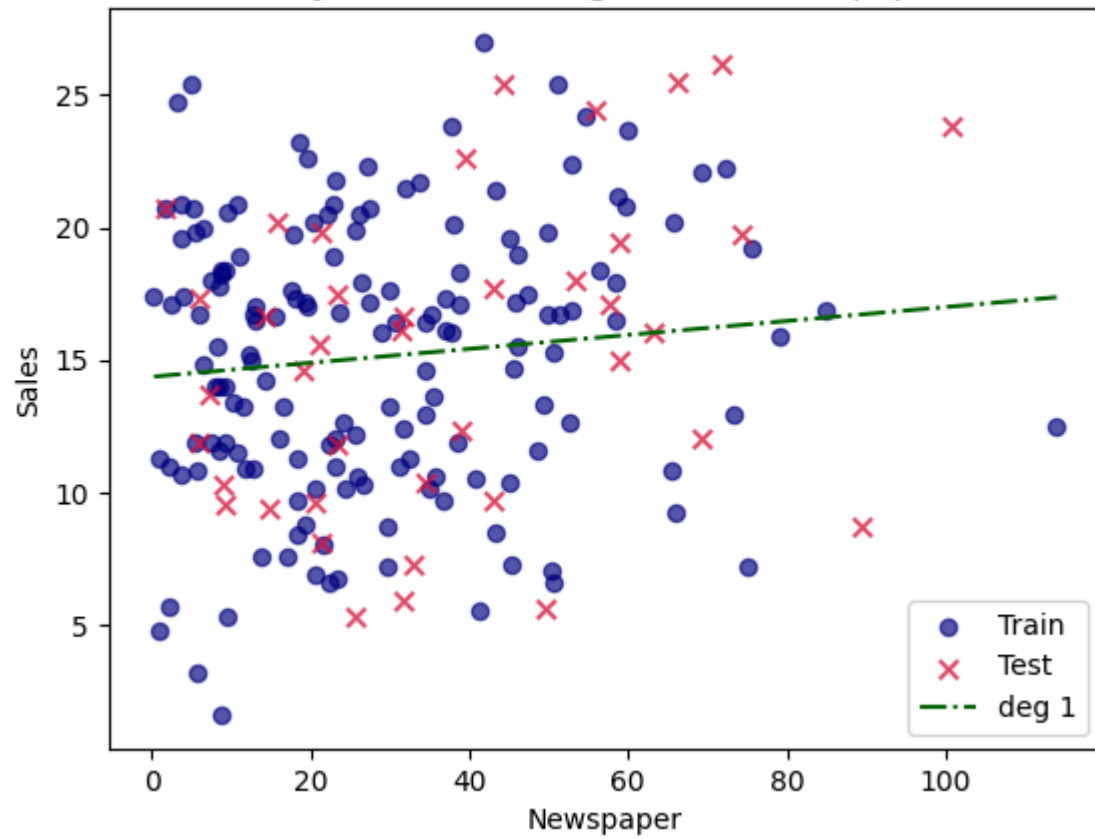
Degree 6: Train=11.7984, Test=13.7516

Degree 7: Train=11.7939, Test=13.7443

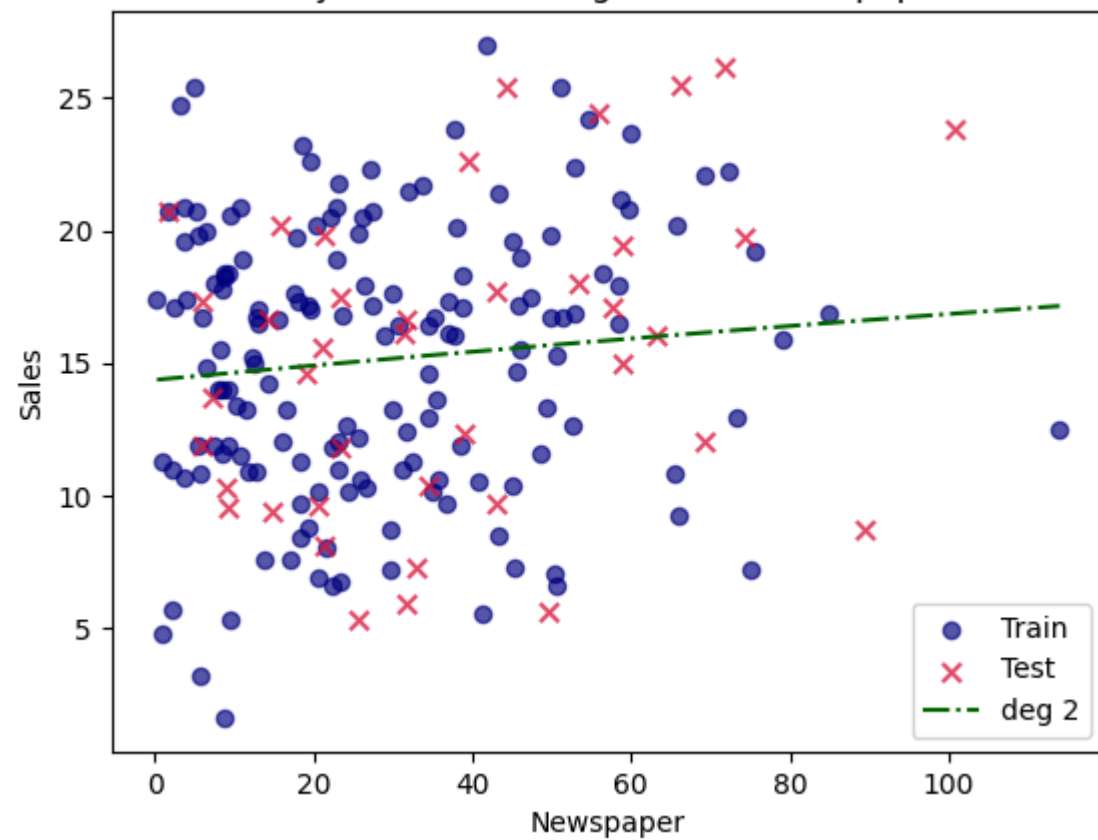
Degree 8: Train=11.7931, Test=13.7269

Degree 9: Train=11.7941, Test=13.7080

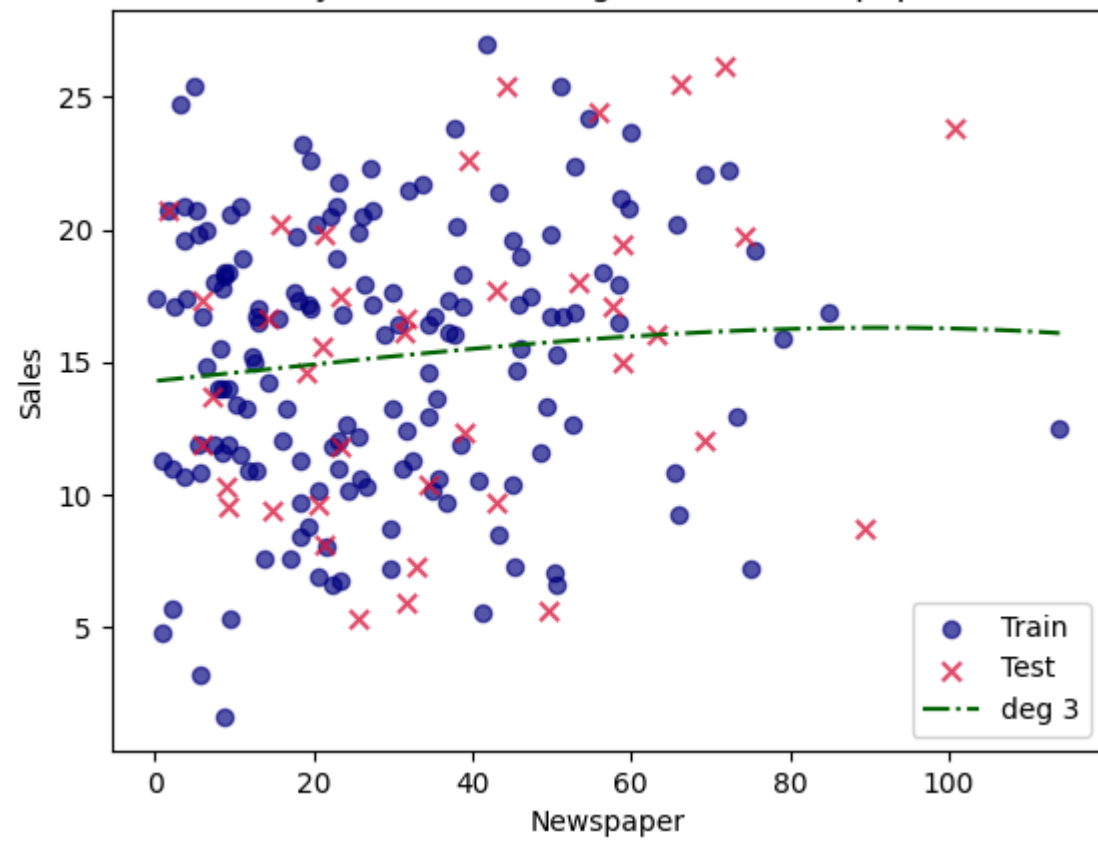
A scatter plot illustrating the training and testing data for a linear classifier. The plot shows two classes of data points: 'Train' (blue circles) and 'Test' (red 'x' marks). A green dash-dot line represents the linear decision boundary (deg 1). The training data points are densely clustered, while the test data points are more sparsely distributed, particularly on the right side of the plot. The decision boundary is a straight line with a slight positive slope, separating the two classes.



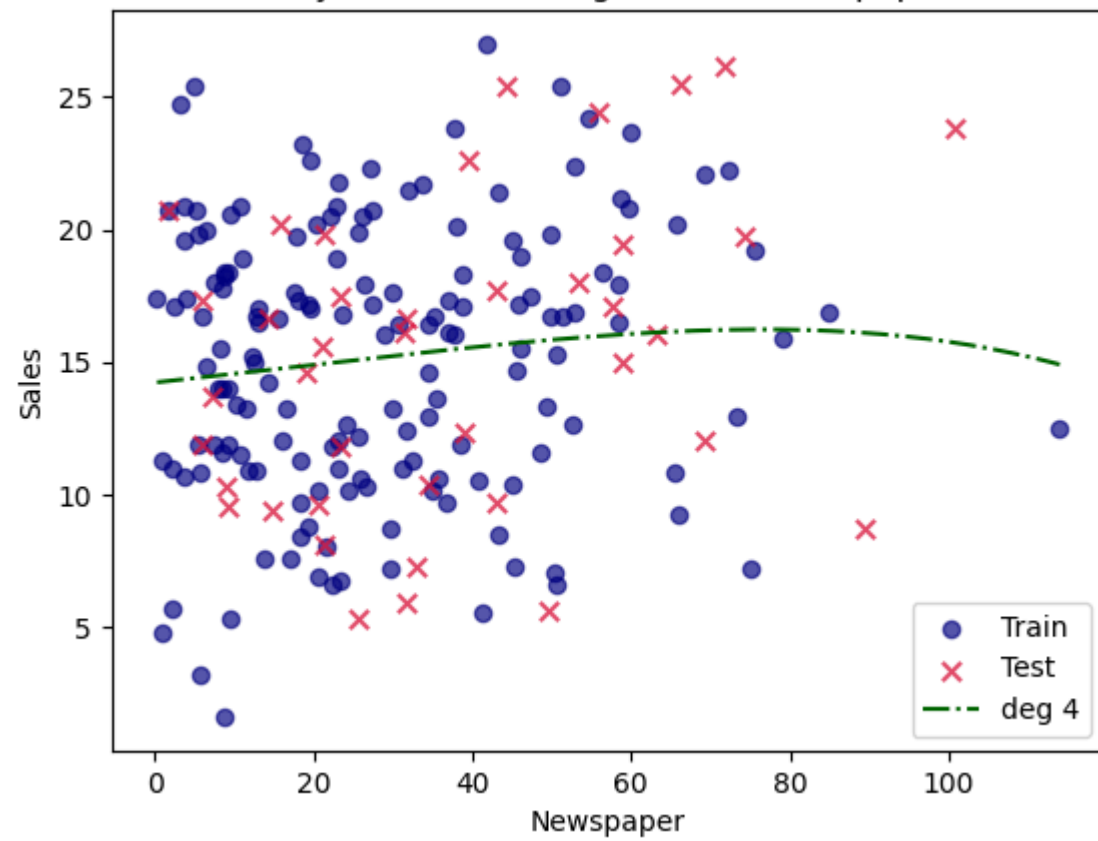
Polynomial Fit of degree 2 for Newspaper



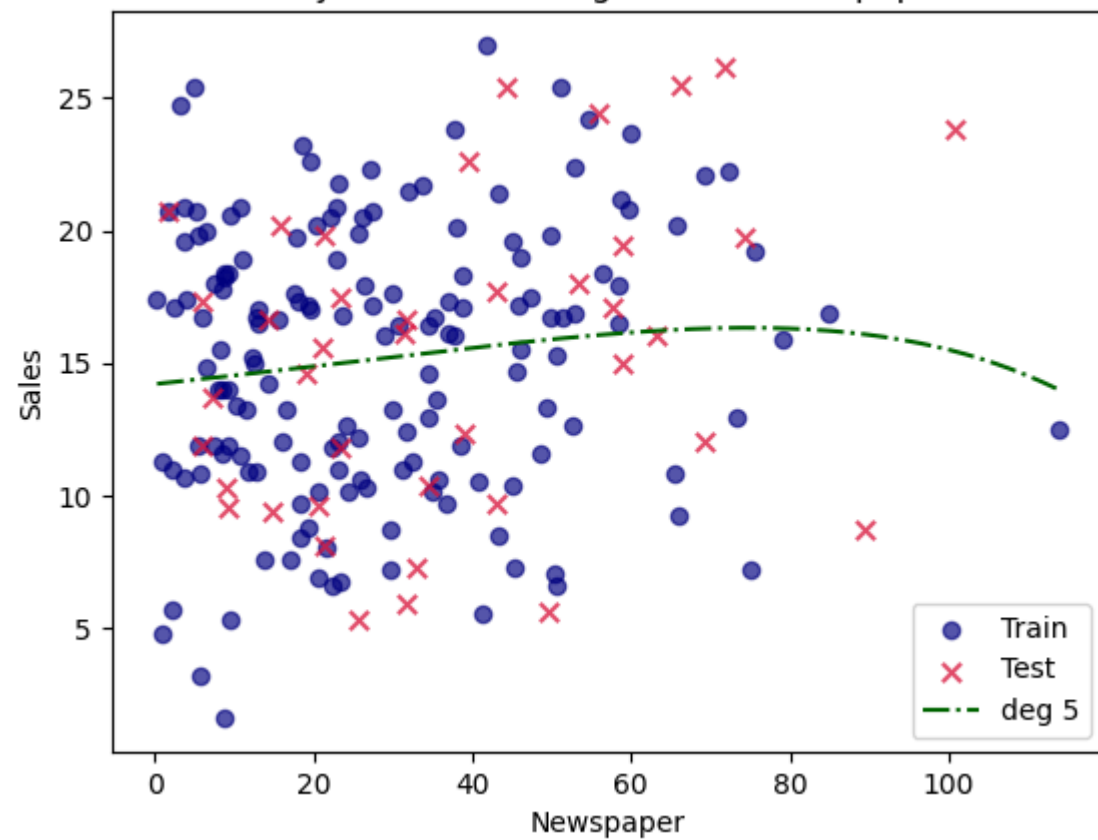
Polynomial Fit of degree 3 for Newspaper



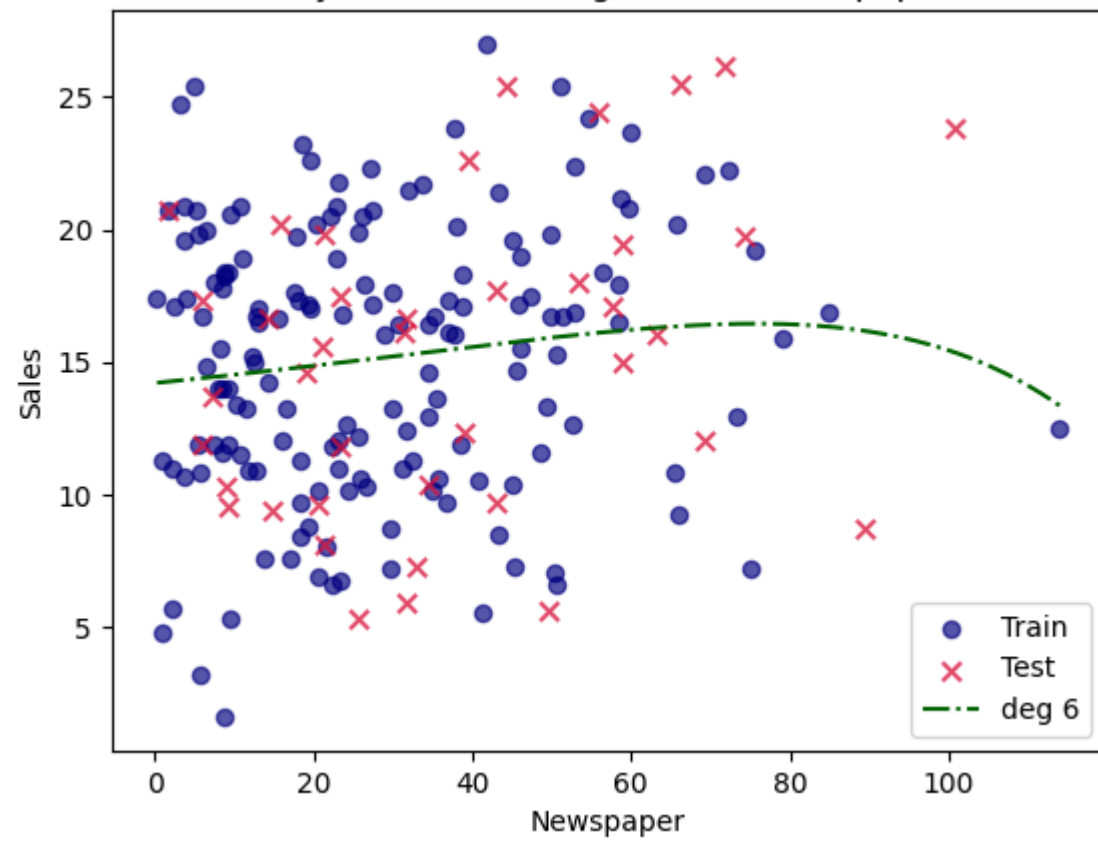
Polynomial Fit of degree 4 for Newspaper



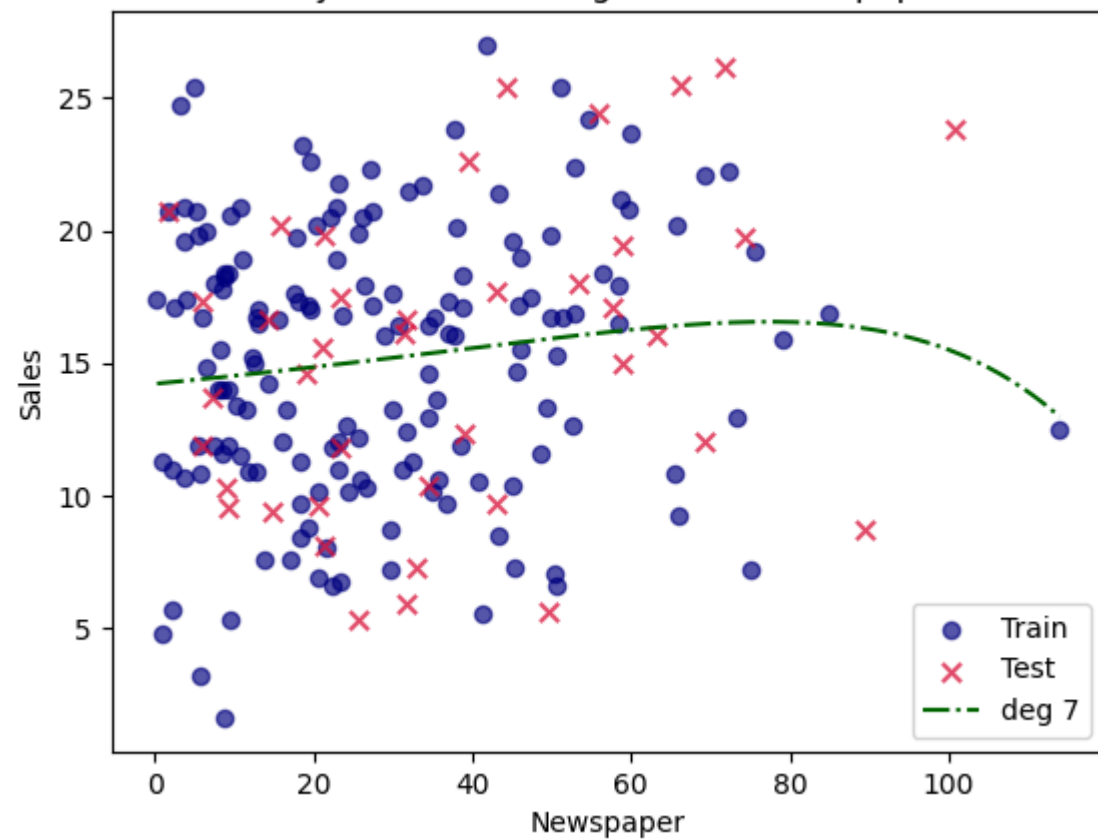
Polynomial Fit of degree 5 for Newspaper



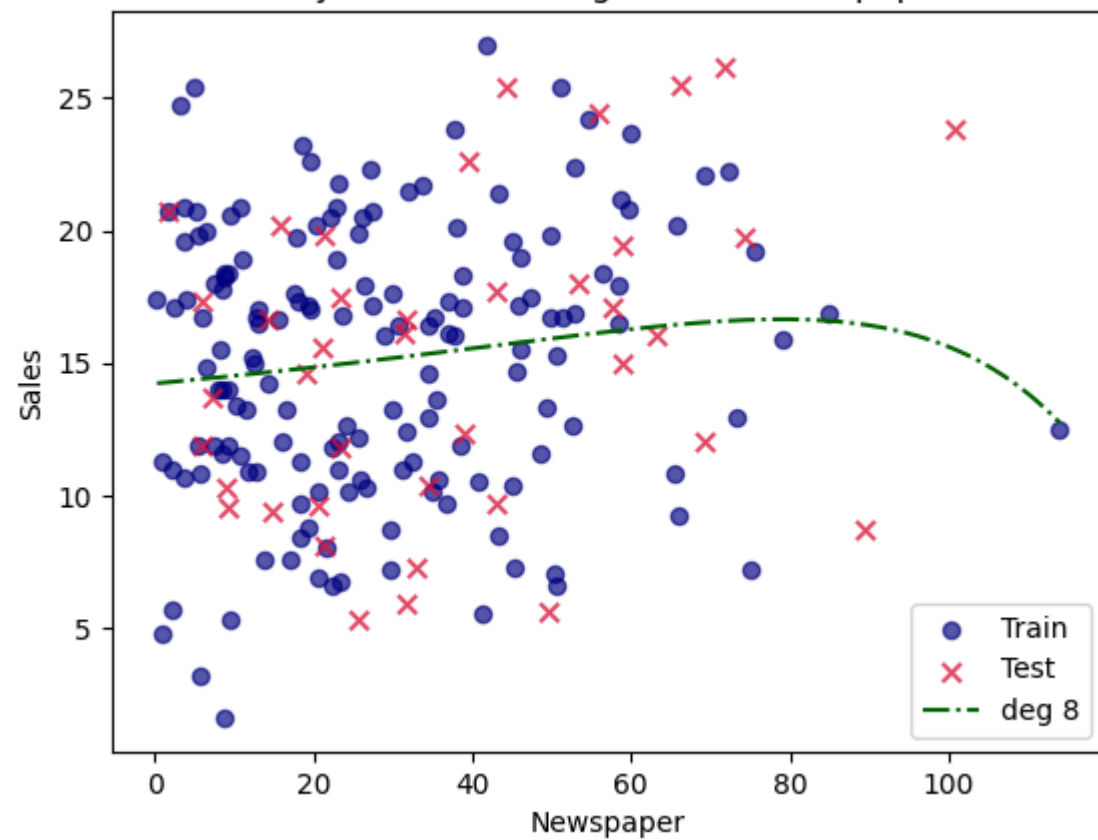
Polynomial Fit of degree 6 for Newspaper



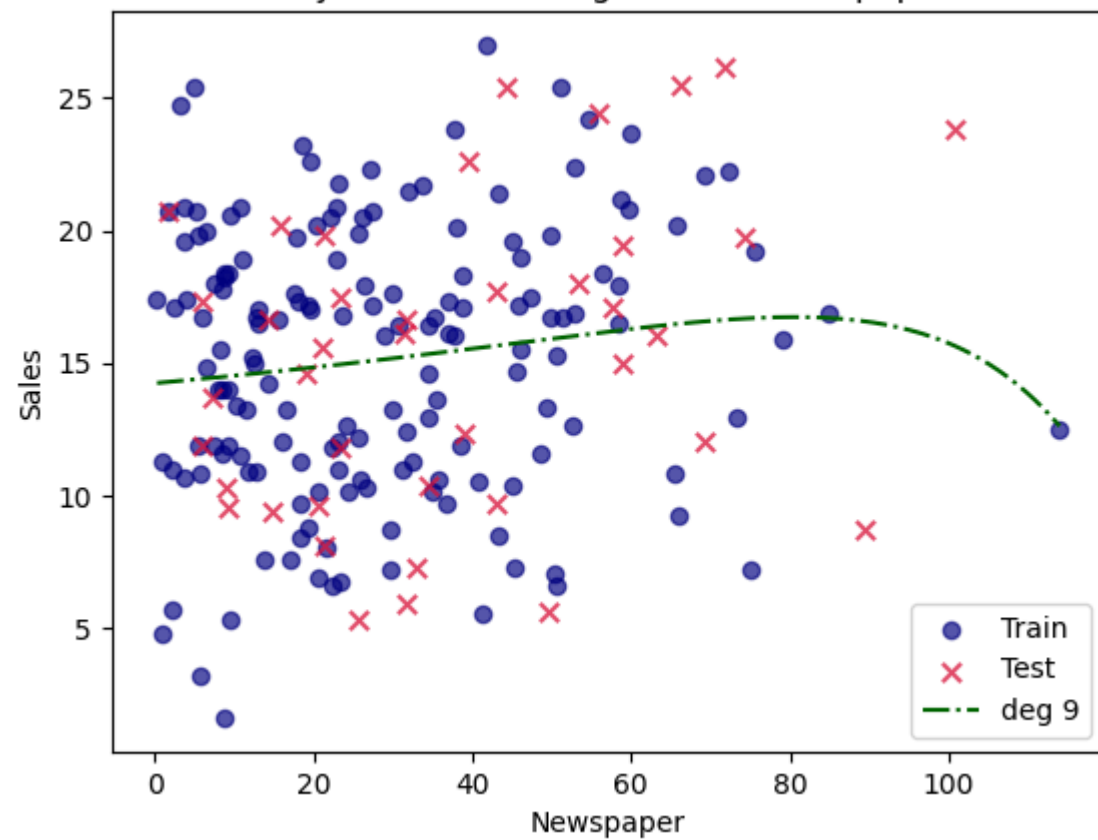
Polynomial Fit of degree 7 for Newspaper

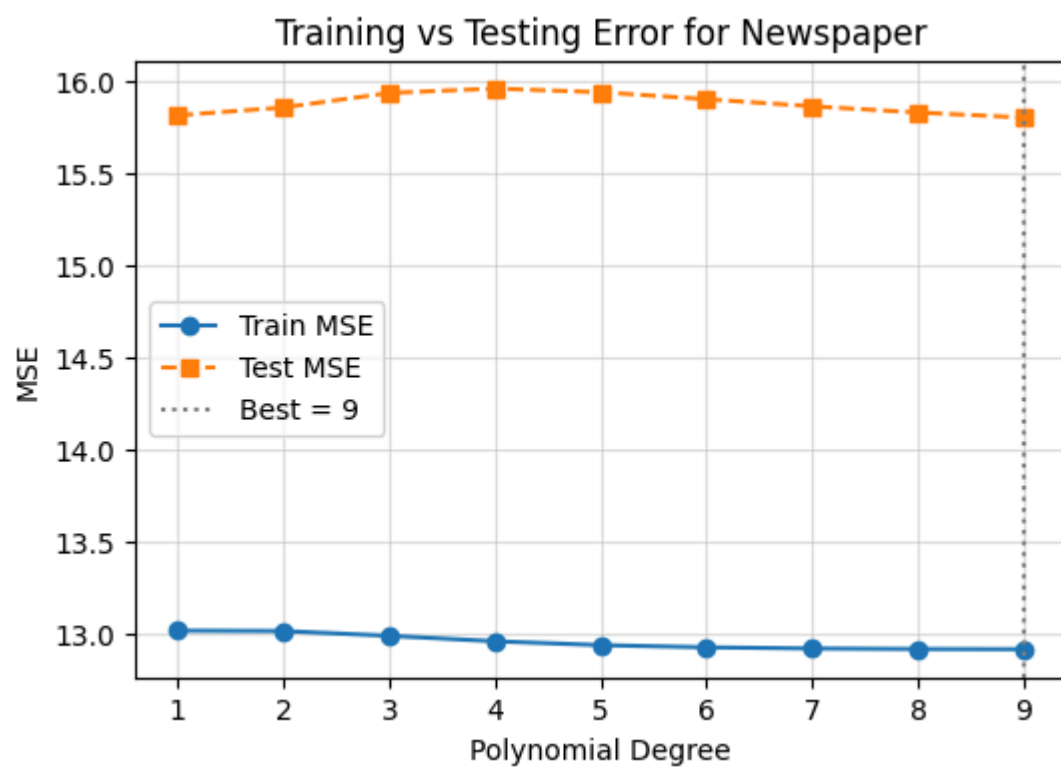


Polynomial Fit of degree 8 for Newspaper



Polynomial Fit of degree 9 for Newspaper





Results for Newspaper:

Degree 1: Train=13.0198, Test=15.8103

Degree 2: Train=13.0176, Test=15.8527

Degree 3: Train=12.9918, Test=15.9313

Degree 4: Train=12.9622, Test=15.9551

Degree 5: Train=12.9414, Test=15.9352

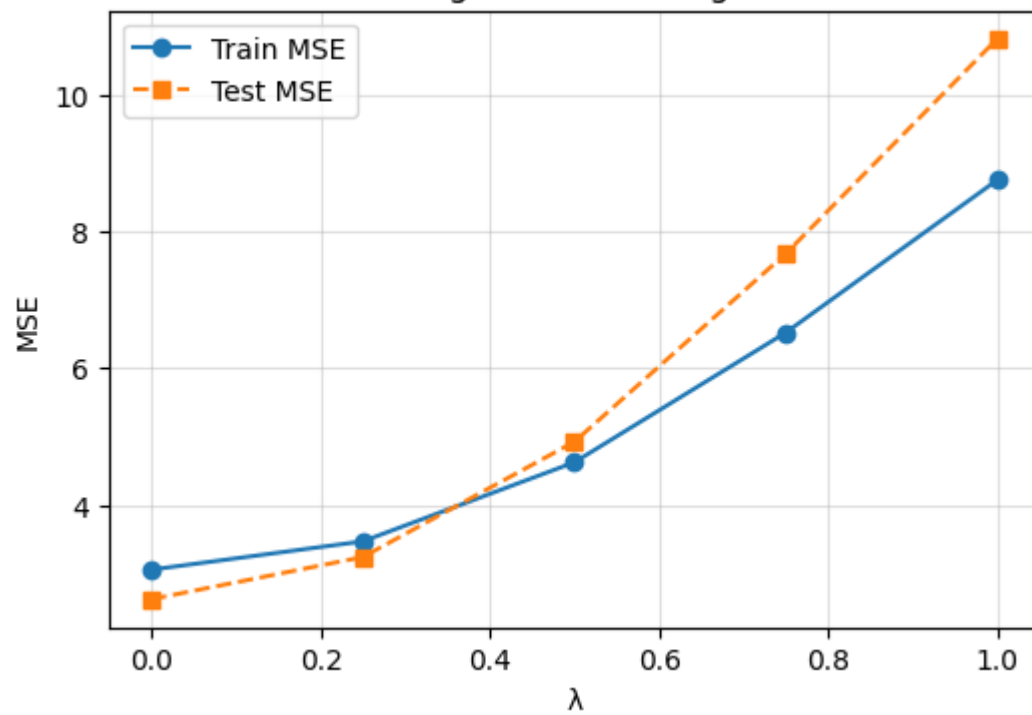
Degree 6: Train=12.9294, Test=15.8976

Degree 7: Train=12.9232, Test=15.8589

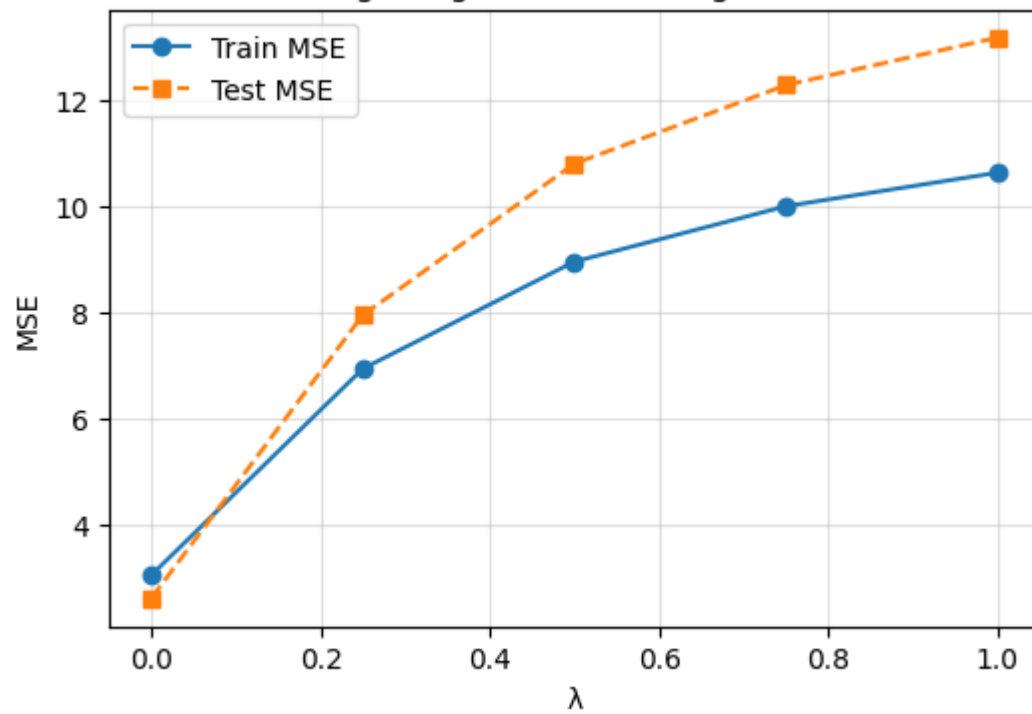
Degree 8: Train=12.9202, Test=15.8254

Degree 9: Train=12.9190, Test=15.7981

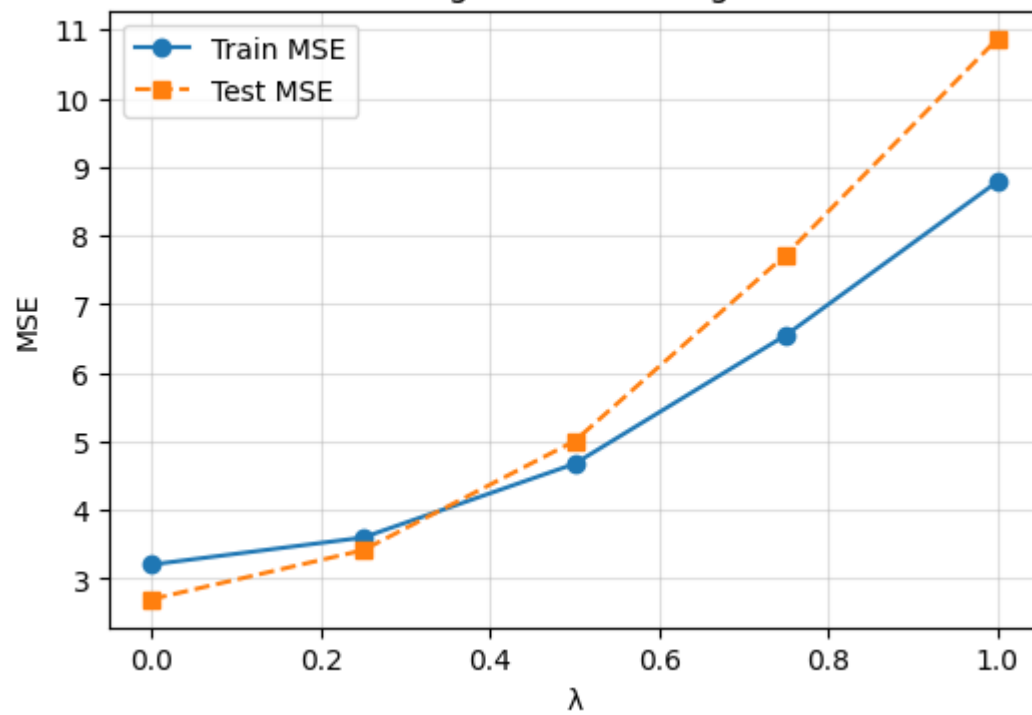
Lasso Regularization (degree=2)



Ridge Regularization (degree=2)



Lasso Regularization (degree=3)



Ridge Regularization (degree=3)

